

LAZYTEST: REPOSITORY-AWARE LLM UNIT-TEST GENERATION FOR PYTHON IN A CONTROLLED THEALGORITHMS CASE STUDY

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Ilya Maltsev
Supervisor: Dr. Suzanne Embury

Department of Computer Science

Contents

Abstract	9
Declaration	11
Copyright	12
Acknowledgements	13
1 Introduction	14
1.1 Motivation and Context	14
1.2 Problem Statement and Scope	15
1.3 Aim and Objectives	16
1.4 Main Contributions	17
1.5 Research Questions	18
1.6 Success Criteria and Evaluation Strategy	19
1.7 Report Structure	20
2 Background and Theory	22
2.1 Automated Unit-Test Generation and the Oracle Problem	22
2.2 Python-Specific Challenges for Generated Tests	23
2.3 Search-Based Software Testing and Pynguin	24
2.4 LLM-Based Unit-Test Generation	25
2.5 Repository Context and Import Handling	26
2.6 Prompt Variants and Generation Routes	27
2.7 Execution-Aware Repair	28
2.8 Evaluating Generated Tests	29
2.9 CI/CD and Pull-Request-Facing Test Generation	30
2.10 TheAlgorithms as a Controlled Case-Study Setting	31

2.11	Summary and Research Gap	32
3	Methodology and Design	33
3.1	Controlled Case-Study Framing and System Boundary	34
3.2	Target Discovery and Evaluation Sets	34
3.3	Generation Conditions	36
3.4	Repository-Aware Context Construction and Import Handling	37
3.5	Bounded Generate–Validate–Repair Loop	38
3.6	Pull-Request Workflow Methodology	39
3.7	Structural Coverage Evaluation	40
3.8	Mutation-Score Evaluation	40
3.9	Metrics and Analysis	41
3.10	Methodological Limitations	43
3.11	Chapter Summary	44
4	Implementation	45
4.1	Pull-Request-Facing GitHub Actions Workflow	45
4.2	Local Repository-Aware Generation Pipeline	48
4.3	Generic Target-File Baseline	50
4.4	Structural Coverage Recalculation	51
4.5	Mutation-Score Evaluation	52
4.6	Artefacts, Reproducibility and Implementation Constraints	53
5	Evaluation and Results	55
5.1	Main Pipeline Comparison	56
5.2	Repair Effectiveness and Executability	58
5.3	Coverage and Mutation Score	58
5.4	Secondary Backend and Prompt Results	59
5.5	Effect of Mutation-Aware Prompting	59
5.6	Local and Quantised Route Analysis	60
5.7	Failure Modes and Denominator Effects	63
5.8	Answering the Research Questions	64
5.9	Chapter Summary	65
6	Conclusion and Future Work	66
6.1	Overview	66
6.2	Answers to the Research Questions	67

6.3	Contributions and Achievements	69
6.4	Critical Reflection and Limitations	70
6.5	Future Work	71
6.6	Final Conclusion	72
Bibliography		73
A Evaluation Artefacts and Supplementary Results		77
A.1	Evaluation Source Files and Aggregation Provenance	77
A.2	Metric Definitions	79
A.3	Route, Model and Prompt Label Mapping	81
A.4	Main Matched Route Source Values	83
A.5	All Route Summary	83
A.6	All Route Denominator Summary	86
A.7	Statement and Branch Coverage	88
A.8	Prompt-Variant Robustness Across Qwen Routes	88
A.9	Matched Full and AWQ Qwen Route Summary	89
A.10	Runtime and Repair-Attempt Evidence	90
A.11	Coverage Rerun Failure Summary	90
A.12	Mutation Denominator and Primary Mutation Outcomes	91
A.13	Failure-Mode Classification Rules	92
A.14	Generation Failure-Mode Counts	93
A.15	Chapter 5 Figure Source Data	93
A.16	Aggregation Caveats	94
B Implementation Artefacts		96
B.1	Implementation Artefact Index	96
B.2	GitHub Actions ChatOps Workflow	97
B.3	Target Discovery and AST-Based Context Extraction	99
B.4	Deterministic Import Handling and Output Sanitisation	101
B.5	Bounded Generate–Validate–Repair Loop	103
B.6	Generic Target-File Baseline	105
B.7	Structural Coverage Recalculation	107
B.8	Mutation Evaluation	109
B.9	Mutation Timeout Retry	112
B.10	Gemini Backend	113

B.11 Route and Prompt Configuration Variants	115
B.12 Implementation Evidence Summary	116

Word Count: 14987

List of Tables

3.1	Evaluation sets and denominators	35
4.1	Implemented LazyTest components	47
4.2	Deterministic import preamble steps	49
4.3	Main implementation artefacts	53
5.1	Main LazyTest and baseline comparison	56
5.2	Evaluation denominators for the main matched routes	56
5.3	Recorded generation time and repair-attempt summary	58
5.4	Selected secondary routes	59
5.5	Qwen 27B prompt variants	61
5.6	Matched Qwen and AWQ comparison	62
5.7	Generation failure modes for the main matched routes	63
A.1	Chapter 5 source-data provenance	78
A.2	Chapter 5 metric definitions	80
A.3	Chapter 5 route-label mapping	82
A.4	Main matched route source values	83
A.5	All route compact metric summary	84
A.6	All route denominator summary	87
A.7	Statement and branch coverage for the main matched routes	88
A.8	Prompt-variant robustness across Qwen routes	89
A.9	Matched full and AWQ Qwen route summary	90
A.10	Runtime and repair-attempt evidence	90
A.11	Coverage rerun failure summary	91
A.12	Mutation denominator and primary mutation outcome summary	91
A.13	Generation failure-mode classification rules	92
A.14	Diagnostic generation failure modes for the main matched routes	93
A.15	Chapter 5 figure source data	94

A.16 Aggregation caveats relevant to Chapter 5	95
B.1 Implementation artefact index	97

List of Figures

4.1	LazyTest implementation architecture	46
5.1	Main pipeline comparison	57
5.2	Selected secondary routes	60
5.3	Qwen 27B prompt variants	61
5.4	Matched full vs AWQ Qwen comparison	62

Abstract

Automated unit-test generation can reduce the manual effort involved in code testing, but generated tests are only useful if they can be executed, measured and interpreted within the target repository. This dissertation develops and evaluates LazyTest, a CI-integrated, execution-aware pipeline for automated Python unit-test generation. The project focuses on repository-aware context extraction, deterministic import handling and bounded pytest-based repair for Large Language Model (LLM)-generated pytest tests.

LazyTest was implemented as two related artefacts. The first is a pull-request-facing GitHub Actions workflow that allows automated test generation to be requested, validated, reported and optionally committed within a repository workflow. The second is a separate local empirical evaluation pipeline used to generate controlled experimental data. The empirical evaluation is framed as a controlled case study on TheAlgorithms/Python, a favourable Python repository with many algorithmic pure-Python modules. This scope is deliberate: the work does not claim to demonstrate generalisation to dependency-heavy or compiled-extension repositories.

The evaluation compares generic target-file generation with repository-aware LazyTest generation and also reports selected local, API-based and search-based routes where comparable metrics were available. Generated tests were assessed using initial pass rate, final pass rate after bounded repair, repair improvement, coverage, viable mutation score, runtime evidence and diagnostic failure modes. The highest-performing repository-aware Qwen route passed 1,106 of 1,144 selected target files, while the main non-repeated Qwen/Qwen3.5-27B repository-aware route passed 1,088 targets. The evaluated Gemini API backend passed 1,037 targets when used inside the same repository-aware pipeline. In the main matched comparisons, repository-aware LazyTest generation achieved higher full-target-set executability than the corresponding generic target-file baselines. Mutation testing provided additional evidence of fault-revealing

ability, with main LLM routes producing viable mutation scores of approximately 66–74%, although these scores apply only to the mutation-evaluated subset.

The results show that, under favourable repository conditions, the combined repository-aware LazyTest pipeline was associated with more executable and measurable LLM-generated pytest tests. However, generated tests remain best understood as regression-oriented or characterisation tests rather than proofs of correctness. The dissertation therefore contributes both a working implementation and an evidence-based account of its limitations, motivating future work on harder repositories, finer-grained ablation, stronger oracle support and maintainer-facing evaluation.

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Acknowledgements

Firstly, I would like to express my sincere appreciation to my project supervisor, Dr Suzanne Embury. Her guidance and support have been instrumental throughout this project. She provided thoughtful and timely advice, as well as detailed suggestions on how the work could be structured and developed. At points when the project was difficult to move forward, her guidance helped me regain a clear sense of direction. Many of the ideas that shaped the final form of this dissertation developed through discussions with her, and I am very grateful for the time, patience and effort she devoted to supervising this work.

I would also like to thank my friends Julius Vander Arend, Serhii Tupikin and Yevhenii Voropaiev. Over the course of the year, we had many discussions about the project. These conversations helped me clarify my thinking, test early ideas and consider technical problems from different perspectives. Their interest, suggestions and encouragement made the process more productive and less isolating, and each of them contributed in some way to how the project evolved.

I am particularly grateful to Julius Vander Arend for allowing me to use his personal computer, which was equipped with an NVIDIA GeForce RTX 5090 graphics card. Access to this machine made it possible to run computationally intensive local large language model experiments and allowed me to collect substantially more data than would otherwise have been possible.

Lastly, I would like to thank my family for their constant support throughout my education, and especially during the demanding final stages of this project. Their encouragement, patience and advice have been a significant source of strength. Without their continued support, I would not have had the opportunity to study here or complete this dissertation.

Chapter 1

Introduction

1.1 Motivation and Context

Unit tests are a standard component of modern software engineering practice. They allow developers to check whether code behaves as expected, detect regressions, document expected behaviour and make changes with reduced risk. While continuous integration allows for more frequent execution of these tests, creating effective unit tests continues to require significant manual effort from developers. Developers still need to select appropriate input values, construct fixtures, define the expected outputs, and update tests as the codebase evolves [7, 9].

Automated test generation aims to reduce this burden by producing tests programmatically. Search-Based Software Testing (SBST) is one established approach: it typically optimises for structural measures such as statement or branch coverage, and Python tools such as Pynguin follow this general coverage-oriented approach [15]. However, optimising for structural coverage alone does not necessarily result in high-quality testing. A generated test suite may cover many lines of code while still containing weak assertions, brittle regression checks, or tests that are difficult for developers to understand and maintain [10, 6, 8].

Large Language Models (LLMs) represent another approach to automating unit-test generation. LLMs can use source code, variable names, docstrings and example usage when creating tests [22]. They may therefore be useful for Python functions that have relatively clear input-output behaviour. However, LLMs also introduce specific failure modes. They may produce syntactically plausible tests that fail during collection or execution because of inappropriate imports, incorrect function signatures, unsupported helper objects, or assertions that are not justified by the source code [22, 29].

These problems are especially important in languages such as Python, where package organisation, runtime context, optional dependencies and import behaviour can determine whether a generated pytest file is executable at all.

This dissertation is motivated by the gap between plausible-looking generated tests and tests that are executable, measurable and practically interpretable in a repository setting. The primary question is not simply whether an LLM can write pytest code in isolation, but whether a pipeline can supply enough bounded repository context for generated tests to respect the real symbols, imports and execution environment of the target module. This dissertation therefore focuses on repository-aware context extraction, import handling and execution-aware repair for LLM-based Python unit-test generation.

1.2 Problem Statement and Scope

The problem addressed in this dissertation is that LLM-generated unit tests for Python can appear syntactically plausible while still being unusable in a real repository. In particular, generated tests may import the wrong module, call unsupported functions or signatures, rely on missing helper objects, or pass without checking meaningful behaviour. The project therefore investigates whether bounded repository-aware context, controlled import handling and execution-aware repair can improve the executability and measurability of generated pytest tests within a controlled Python repository case study.

This dissertation describes the evaluation of LazyTest, a CI-integrated automated unit-test generation pipeline for Python. LazyTest consists of two related layers. The first layer is a GitHub Actions workflow that operates on pull requests, allowing maintainers or authorised users to request, validate, report on and upload generated test artefacts. Where configured, generated tests can also be committed back to the original pull-request branch. The second layer is an empirical evaluation pipeline that generates, validates, repairs and evaluates candidate tests within controlled experimental settings (see Chapter 4, Figure 4.1, and Appendix B). Both layers are discussed further in Chapter 4.

TheAlgorithms/Python is the focal point of the empirical evaluation [25]. It was selected because it provides comparatively favourable evaluation conditions for an initial case study: its contents are broadly consistent with algorithmic modules that often accept inputs, perform calculations or data manipulations, and return outputs.

Its contribution guidance also encourages documentation conventions including type hints, docstrings and doctests [24]. That said, this does not mean every file within this repository is simple, self-contained or fully documented. Chapter 3 outlines the target-discovery method used for this repository, as well as the evaluation conditions and configurations.

This scope is intentional. TheAlgorithms is not treated as representative of all Python repositories. Results from TheAlgorithms/Python would not by themselves show that LazyTest generalises to larger repositories with many external dependencies, web applications, complex fixtures, or compiled extensions. Larger repositories such as Requests, Pandas and NumPy are therefore treated as future generalisation targets rather than as completed empirical evaluation subjects in this dissertation, unless supported by separate evidence (see Chapter 5 and Appendix Table A.16).

This problem is also affected by the test oracle problem, so caution is required when interpreting the results of generated tests. In many cases, there is no independent oracle proving the intended semantic correctness of the target code. A passing generated test can show that a particular behaviour is exercised and preserved for a given input, and mutation testing can provide comparative evidence of fault-revealing ability. However, generated tests should not be relied upon as definitive proof of correct program behaviour [3].

1.3 Aim and Objectives

The aim of this project is to examine, through a controlled case study on TheAlgorithms/Python, whether a CI-integrated and execution-aware LLM pipeline can generate executable pytest unit tests and provide measurable indicators of their structural coverage and fault-revealing ability. The objectives are:

Objective 1: CI/CD Integration – Implement a GitHub Actions workflow that allows authorised pull-request users to request generated tests, validate them with pytest, report the outcome, upload artefacts and optionally commit generated tests back to the pull-request branch.

Objective 2: Generation Routes – Implement and evaluate the generation routes used in LazyTest, including Pynguin where comparable metrics are available, local open-weight LLM routes, a generic target-file LLM baseline and an API-based LLM backend inside the same repository-aware LazyTest

pipeline. The aim is to compare these routes under a shared metric schema where complete data are available, rather than to produce a universal model leaderboard (see Listing B.9 and Appendix Table A.3).

- Objective 3:** Controlled Evaluation on TheAlgorithms – Construct a local empirical evaluation pipeline for measuring the executability, repair success, structural coverage, mutation score, runtime overhead and failure modes of generated pytest tests on TheAlgorithms/Python under reproducible conditions.
- Objective 4:** Repository-Aware Context Extraction – Use source-code analysis, including AST-based symbol extraction, to constrain generation to actual target functions and classes as defined in the repository. The aim is to reduce unsupported target-symbol use and avoid relying on the model to infer the available API unaided (see Listing B.2).
- Objective 5:** Import Handling – Address import-related failure modes through deterministic preambles, controlled module loading and restrictions on model-generated imports (see Listing B.3).
- Objective 6:** Execution-Aware Repair – Develop a bounded pytest-based repair loop that uses execution failures and tracebacks as feedback, with the aim of increasing validation success while keeping repair attempts reproducible and limited (see Listing B.4).
- Objective 7:** Test-Quality Evaluation – Assess generated tests using initial pass rate, final pass rate after bounded repair, repair improvement, structural coverage, mutation score on the mutation-evaluated target set, runtime overhead and failure-mode analysis (see Chapter 5 and Appendix Tables A.2, A.5 and A.12).

These objectives concentrate the project on producing generated tests that are executable and can be assessed in a repository setting. They do not propose solving automated test generation for all Python projects.

1.4 Main Contributions

This dissertation makes four main contributions.

First, it presents LazyTest as a pull-request-facing automated test-generation workflow implemented through GitHub Actions. This contribution shows how generated tests can be requested, validated and reported within a pull-request review process, rather than requiring maintainers to run separate local scripts outside the repository workflow (see Chapter 4 and Listing B.1).

Second, it creates a repository-aware LLM generation pipeline for Python unit tests. Rather than relying only on a generic target-file prompt, the pipeline uses controlled context construction, AST-extracted symbols and deterministic import handling to address common causes of non-executable generated tests, including unsupported target symbols, incorrect imports and inconsistent module loading (see Listings B.2 and B.3).

Third, it constructs a bounded Generate–Validate–Repair loop in the local empirical evaluation pipeline. Generated tests are executed using pytest, and failures during execution serve as feedback for later repair attempts within a fixed retry and timeout budget. The aim is to improve generated-test executability while keeping the repair process reproducible and finite (see Listing B.4).

Fourth, it establishes a controlled TheAlgorithms/Python case-study evaluation and reports the resulting evidence. The evaluation assesses generated tests through initial pass rate, final pass rate after repair, structural coverage, mutation score on the mutation-evaluated target set, runtime overhead and categorised failure modes (see Chapter 5 and Appendix Tables A.5 and A.2). In the evaluated results, the highest-performing repository-aware Qwen route passed 1,106 of 1,144 selected target files, and the main non-repeated Qwen 27B repository-aware route passed 1,088 targets, as reported in Chapter 5 and Appendix Table A.5. The contribution is therefore a focused evidence-based analysis under bounded repository conditions, not a broad benchmark across Python ecosystems.

1.5 Research Questions

The project is guided by six research questions:

- RQ1:** How effectively can LazyTest generate executable pytest unit tests for TheAlgorithms/Python within a controlled case study?
- RQ2:** To what extent does bounded execution-aware repair increase the proportion of generated test suites that pass validation compared with initial generation?

- RQ3:** How do a generic target-file generation condition and repository-aware LazyTest generation differ in initial pass rate, final pass rate after bounded repair, structural coverage, mutation score and failure modes?
- RQ4:** For the mutation-evaluated target set, to what extent do generated tests exhibit fault-revealing behaviour, measured through mutation score, rather than simply increasing structural coverage?
- RQ5:** What limitations remain in import handling and repository-context understanding after repository-aware prompting and bounded repair, even within a pure-Python repository with comparatively favourable algorithmic structure?
- RQ6:** Where complete comparable data are available, how do the evaluated generation backends and routes, including local open-weight LLMs and an API-based Gemini backend, differ in reliability, runtime overhead and generated-test quality within this controlled setting?

RQ6 is secondary. The main contribution is the pipeline and its repository-aware mechanisms, not a comprehensive model leaderboard. Backend and route comparisons are included in the main quantitative comparison only where complete pass-rate, coverage, mutation, runtime and failure-mode data are available. Otherwise, they are treated as exploratory observations (see Chapter 5 and Appendix Table A.16).

1.6 Success Criteria and Evaluation Strategy

The project is evaluated at two levels.

At the implementation level, LazyTest was assessed against the operation of both intended layers. The implementation success criteria were:

- the pull-request-facing workflow parses an authorised request, identifies changed non-test Python files, runs a selected generation route, validates generated tests, uploads artefacts and reports results to maintainers;
- the local empirical pipeline constructs bounded repository context, generates pytest files, validates them, performs bounded repair where configured, and logs outcomes for later analysis;
- the generated artefacts and logs are sufficient to support the pass-rate, structural coverage, mutation, runtime and failure-mode analyses reported in Chapter 5.

These implementation mechanisms are described in Chapter 4, Figure 4.1, and Appendix B.

At the evaluation level, the project measures whether generated tests are executable and whether they provide measurable indicators of generated-test quality. The primary metrics for the main evaluated conditions are initial pass rate, final pass rate after bounded repair, repair improvement, structural coverage, mutation score on the mutation-evaluated target set, runtime overhead and failure modes (see Chapter 5 and Appendix Tables A.2, A.5 and A.14). Mutation score is interpreted comparatively as evidence of fault-revealing ability, not as proof of correctness [19, 20, 11]. Repair improvement is interpreted as an executability and validation-success measure, not as direct proof that repaired tests are semantically stronger.

This evaluation strategy reflects a central problem in automated test generation: tests that execute code are not necessarily strong tests. Pass rate is a necessary metric because non-executable tests have little practical regression value, but it is not sufficient. Structural coverage shows what code was exercised, but it can reward tests that execute code without meaningful assertions [10]. Mutation score and failure-mode analysis therefore provide complementary evidence about fault-revealing behaviour and the causes of failure.

All results are interpreted with respect to TheAlgorithms/Python case-study scope. The evaluation is used to assess LazyTest under the comparatively favourable conditions of a mostly pure-Python algorithm repository and to identify barriers that would need to be addressed before applying the approach to more complex repositories (see Chapter 5 and Appendix Table A.16).

1.7 Report Structure

Chapter 2 presents the background and theory for the project, including the oracle problem, SBST, Pynguin, LLM-based test generation, execution-aware repair, repository-context challenges, and coverage versus mutation testing.

Chapter 3 defines the methodology. It explains the controlled TheAlgorithms/Python case-study design, target-discovery procedure, generation conditions, metrics, and the distinction between main and exploratory comparisons.

Chapter 4 presents the implementation of LazyTest, including the GitHub Actions

workflow, local evaluation pipeline, generation routes, Gemini API backend, AST-based context extraction, deterministic import handling, validation, repair and logging.

Chapter 5 reports and analyses the evaluation results, including pass rates, repair improvement, structural coverage, mutation score, runtime overhead, denominator effects and failure categories.

Chapter 6 concludes the dissertation by answering the research questions, reflecting on limitations, and outlining future work towards more dependency-heavy Python repositories.

Chapter 2

Background and Theory

This chapter presents the technical and research background for the dissertation. It focuses on automated Python unit-test generation, the difficulty of making generated tests executable within a real repository, and the evaluation of those tests through executability, coverage, mutation score and failure-mode analysis. It discusses the test oracle problem, Python-specific failure sources, Search-Based Software Testing (SBST), Pynguin, Large Language Model (LLM)-based test generation, repository-aware context selection, generation-route variation, execution-aware repair, evaluation metrics, and the controlled TheAlgorithms/Python case-study framing used in this dissertation.

2.1 Automated Unit-Test Generation and the Oracle Problem

Unit tests check the behaviour of a function, class or module under controlled conditions. In Python, they are commonly written using frameworks such as pytest, where each test prepares inputs, executes the target code and asserts expected behaviour. Automated unit-test generation attempts to produce such tests programmatically, but the generator must select inputs, call the correct functions or methods, construct required objects, handle imports and decide what to assert. A generated test can therefore be executable but weak, or logically plausible but unusable because it imports the wrong module or calls a non-existent function.

The practical motivation comes partly from the cost of manual testing. Empirical work shows that testing is not performed uniformly or continuously, and that test

creation and maintenance compete with other development tasks [4]. Continuous integration can make execution cheaper and more frequent, but it does not remove the work of designing useful tests [9]. Automatically generated tests can complement manual testing, while still raising questions about readability, maintainability and fault detection [23].

A central reason for this difficulty is the test oracle problem. A test oracle is a mechanism for deciding whether a program's output is correct for a given input. In a manually written unit test, the oracle may be an explicit expected value, a known mathematical property, a reference implementation, a formal specification, or a developer's domain knowledge. Automated generation rarely has access to this certainty [3]. A generator may be able to call a function many times, but still not know what result should be expected.

This matters directly for LLM-generated unit tests. A model may infer a plausible expected result from source code, docstrings or examples, especially for deterministic and well-documented code. However, if it mirrors the implementation too closely, the assertion may preserve an existing bug; if it invents an expected result, the test may fail for the wrong reason; and if it avoids strong assertions, the test may pass while checking little.

For this reason, generated tests in this dissertation are interpreted as partial evidence. They are best understood as executable regression-oriented or characterisation tests unless there is an independent oracle. A passing generated test can show that a particular behaviour is exercised and preserved for a tested input; it cannot prove that the behaviour is intended. Mutation testing can provide stronger evidence that a test detects behavioural changes, but even mutation score remains an empirical indicator rather than an absolute measure of correctness [11].

2.2 Python-Specific Challenges for Generated Tests

Python complicates automated unit-test generation through dynamic typing, packaging, imports and runtime behaviour.

In statically typed languages, declared types and method signatures provide useful information before execution. Python offers less certainty. Type annotations may be absent, incomplete or unenforced. Python functions may accept several possible input types, inspect values at runtime, and fail only after traversing a significant part of an execution path. A class may create attributes dynamically. A module may also

run code at import time. These features are useful to developers, but awkward for automated generation [16, 17].

The first problem is input construction. Without reliable type information, a generator may call a function with the wrong kind of object, the wrong number of parameters, or a value that exercises only an irrelevant error path. Type hints, docstrings and doctests can help, but they are not guaranteed to exist or to be complete. Pynquin’s design and later empirical work make this issue visible: optional Python type information can guide generation, but missing or imprecise type information remains a practical obstacle [15, 17].

The second problem is import resolution. A generated pytest file can fail before it reaches a single assertion. Python collection depends on the current working directory, package layout, `sys.path`, installed dependencies, relative imports and project configuration. A test generated from an isolated source file may look sensible while still being impossible to import in the actual repository environment. LLM unit-test-generation studies have observed failures involving missing context, incorrect imports, hallucinated dependencies and invalid calls, while repository-aware test-generation work has explored precise contextual information as one way to improve generated-test executability [22, 29, 28].

The third problem is side effects. Some modules read files, query the network, use randomness, depend on time, print to standard output, or perform expensive computation. A useful test may need fixtures or mocking; if this is missed or overused, tests can become slow, flaky, environment-dependent or too detached from meaningful behaviour. These problems explain why this dissertation does not treat Python test generation as simple code completion: repository context and execution environment both matter.

2.3 Search-Based Software Testing and Pynquin

Search-Based Software Testing frames test generation as an optimisation problem. Instead of asking a human or a model to write tests directly, an SBST tool generates candidate tests, executes them, measures their fitness, and modifies them to improve an objective. That objective is often structural coverage, such as statement or branch coverage [18].

Evolutionary algorithms are common in this area. DynaMOSA, for example, dynamically focuses search effort on currently relevant uncovered targets rather than

treating all coverage targets as static objectives [18]. This makes SBST well suited to structural coverage objectives.

Pynguin adapts automated unit-test generation to Python [15]. It analyses Python modules, generates candidate tests, executes them, and searches for tests that improve structural coverage. For this dissertation, Pynguin matters for two reasons. First, it provides an established non-LLM point of comparison where comparable metrics are available. Second, it shows both the promise and limits of coverage-driven Python test generation.

Those limits are important. SBST can execute code paths without understanding why those paths matter. It may generate regression assertions based on observed values, but these are not necessarily semantically meaningful. Such tests may still be useful as change detectors, but can be difficult for maintainers to interpret.

Generated tests may also suffer from readability and maintainability problems. Prior work shows that test suites can be hard to maintain even when they execute correctly, with problems such as obscure naming, unexplained constants, excessive assertions and brittle reliance on implementation details [6, 8]. SBST therefore motivates this project, but it does not solve the repository-aware import handling, assertion quality or maintainer-readability problems addressed by LazyTest.

2.4 LLM-Based Unit-Test Generation

Large Language Models offer a different route into unit-test generation. They can use function names, docstrings, examples, comments, type hints and surrounding source code, making them relevant where Python functions have relatively clear input-output behaviour [22].

However, generating unit tests is fundamentally different from code generation. Typically, code generation involves translating specifications into implementations. Generating unit tests, on the other hand, is generally an inverse process. The implementation has been created; now the goal is to infer how the implementation should be invoked, what behaviours should be verified, what edge cases are most important, and what assertions can be made [22, 29, 5].

Generated code can appear to be well formed yet semantically incorrect. In unit-test generation, this tends to manifest itself as incorrect imports, artificially introduced helper functions, unsupported fixtures, mis-specified calling signatures, inaccessible attributes, incorrectly anticipated values, or weak assertions [22, 29]. Although the

generated test may appear believable, pytest may still reject it for reasons including collection errors or runtime failures.

Three failure types are particularly relevant to this dissertation. First is collection and import failure: the model may use an inaccessible path, assume the repository is installed as a package, or import an object directly when the execution framework only allows module-level access. These failures occur before the target function is tested.

Second is interface failure. The model may call a real function with the wrong parameters, reference a non-existent attribute, or presume that a function returned a richer object than it actually does. These failures often arise when the model sees only the target source file and must deduce the surrounding invocation context.

Third are weak-passing tests. A test may execute successfully while only checking a non-null result, a type, or a coverage-increasing invocation. Such tests are executable but less useful than stronger tests.

These categories explain why this dissertation uses multiple metrics: initial pass rate, final pass rate after bounded repair, coverage, mutation score and failure-mode analysis. No single metric adequately captures all aspects of performance.

2.5 Repository Context and Import Handling

A central problem in LLM-based test generation is deciding how much repository context to provide. Too little context forces the model to guess; too much can become costly, noisy and difficult to manage.

The simplest approach is target-file-only prompting, where the model is given one source file and asked to generate tests for it. This is a reasonable baseline rather than a straw man, since developers may use LLMs manually in a similar way. It can succeed when the file contains the information needed to invoke the target, imports are straightforward, and expected behaviour is evident in docstrings or examples.

Target-file-only prompting fails in scenarios where real tests cannot simply run against a pasted version of the target file. Instead, real tests run within a repository environment. The model may not know the correct import path, available helper modules, project layout, dependency environment or test conventions. Additionally, a target file may rely upon sibling modules or constants/classes that are not referenced anywhere within the prompt. Even in favourable pure-Python repositories, this can result in both collection and execution failure [22, 29, 28]. Otter addresses a different objective than LazyTest, but provides supporting evidence that artefacts produced by a test generator

may require external verification, review or repair rather than relying exclusively on a model's first response [1].

At the other end of the spectrum is providing the entire repository environment. In theory, this might reduce missing context; in practice, repositories can contain hundreds or thousands of irrelevant, generated, optional or configuration-dependent files. Context windows are limited, and long prompts are not uniformly consumed by models. Work on long-context and repository-level code generation shows that retrieval and context selection still matter even when more tokens are available [13, 30, 14]. The issue is therefore not only context length but context selection: irrelevant files and configuration details can add noise without helping the model construct executable tests.

A more realistic alternative is bounded repository-aware context. This means limiting context to information directly relevant to the target under test: the source file containing the target, public symbols, signatures, docstrings and examples, import path, and controlled execution interface. RATester is highly relevant because it explores repository-aware unit-test generation and uses accurate context injection via language-server-based definition lookup [28]. Although its scope is different from LazyTest, its objectives align closely: generated tests may fail due to lack of project-level context; conversely, indiscriminate expansion of context can introduce excessive irrelevant information.

As additional supporting evidence for the same broad principle, adjacent repository-level code-completion studies provide further justification for using selected cross-file context in repository-level code tasks rather than uncontrolled prompt expansion [30, 14]. This provides the theoretical motivation for LazyTest's repository-aware design (see Chapter 4 and Appendix B, Listings B.2–B.3). Generated tests need sufficient context to be executable, but they should not receive so much context that generation becomes uncontrolled. Bounded context extraction therefore acts as a compromise between target-file-only prompting and full repository ingestion.

2.6 Prompt Variants and Generation Routes

Prompt design affects generated tests. Minor wording variations can influence whether a model produces strict assertions, creates artificial imports, focuses on happy-path cases, or attempts edge-case testing. This is visible in unit-test generation because outputs must satisfy Python syntax, pytest format, valid target invocation, no source

redefinition and valid assertions [22, 29].

The prompts studied here are therefore not merely stylistic variants. A generic prompt requests pytest tests for a target file. A repository-aware prompt can constrain imports, enumerate available functions and classes, use docstrings and discourage unsupported edge cases. A mutation-aware prompt can encourage tests that discover behavioural differences rather than only increasing coverage. Prompt repetition is a milder variant: recent work reports improvements for some non-reasoning LLM outputs in certain benchmark settings [12], but not specifically for unit-test generation, so it is treated only as an evaluated prompt variant.

Additionally, distinct generation routes also require careful interpretation. A generic target-file baseline, a locally hosted open-weight model, an API-based model backend and Pynguin do not constitute identical experimental conditions. Pynguin relies on search-based generation rather than natural-language prompting of an LLM. A locally hosted route allows control over serving configuration and data locality but includes hardware and serving configuration as additional variables. An API-based backend can be integrated into LazyTest’s pipeline, but results will similarly depend on model behaviour, provider settings, output limits and retry handling (see Chapter 4 and Appendix B, Listings B.9 and B.10).

For this dissertation, the key point is comparability. When evaluating an API-based LLM backend as part of the LazyTest pipeline, it should be viewed as a model substitution within LazyTest’s repository-aware architecture rather than as an independent repository-operating tool. It receives the same type of extracted context, deterministic import harness and pytest feedback loop where implemented (see Chapter 4 and Appendix B, Listings B.9 and B.10). Results should therefore be compared as specified pipeline-route configurations, rather than viewed as a universal ranking of model capability.

2.7 Execution-Aware Repair

Static test generation is brittle: a model may generate code that appears reasonable but fails immediately under pytest. Execution feedback changes this interaction by running the generated test, capturing the failure and prompting the model to fix the problem.

This gives rise to a Generate–Validate–Repair (GVR) pattern. The generator produces a candidate test. The validator executes it. If execution fails, the repair step uses

the error message or traceback as feedback. The GVR cycle repeats until the model reaches the end of a specified budget [5, 29, 27]. This creates a compact loop with direct validation evidence, although successful repair still does not prove semantic test strength.

There are other recent systems using similar mechanisms. ChatJUnit validates the generated tests and uses compilation or runtime errors as signals for repair [5]. TestGen-LLM provides adjacent industrial evidence that LLM-generated test artefacts should be filtered through build, reliability and coverage evaluations before providing them to developers [2].

Execution feedback is beneficial because many failures are specific: a traceback may show an ImportError, missing argument, wrong attribute, failed assertion or timeout. The model no longer has to make blind guesses, but can respond to an observed failure. This is especially relevant in Python, where many errors appear only during import or runtime.

Repair has limitations as well. A repair loop can increase executability without increasing test strength. The model may eliminate challenging assertions, avoid a failing path, reduce checks, or replace a meaningful expected value with a looser condition. A final passing test may therefore be more executable than the initial failing test without being stronger.

That distinction affects how LazyTest is evaluated. Improvements due to repair are interpreted as metrics for executability and validation success. They need to be considered together with coverage, mutation score and failure categories. Otherwise, it is possible to design a pipeline that appears successful simply by learning to produce tests that pass without validating much. The repair loop must also be bounded. An unbounded repair loop would be harder to reproduce, harder to compare between environments, and less amenable to evaluation in a CI-like format (see Chapter 4 and Appendix B, Listing B.4).

2.8 Evaluating Generated Tests

Tests produced by generators should be assessed at multiple levels. A test that cannot be collected or run cannot contribute regression value, so pass rate is necessary.

Pass rates alone, however, are insufficient. A generated suite may pass because it avoids meaningful assertions, runs only an easy path, or hard-codes observed outputs

without testing a property. Structural coverage is useful but limited: statement coverage reports executed statements and branch coverage adds control-flow information, but neither verifies assertion quality [10].

Mutation testing provides a complementary view. It introduces small artificial modifications, or mutants, and assesses whether the generated suites detect them. Each mutant is killed if tests fail when the mutated program is executed. Suites that kill more viable mutants provide stronger comparative evidence of fault-revealing capacity than suites that simply execute more lines.

An example mutation score can be defined as follows:

$$MS = \frac{K}{K + S} \times 100\%$$

where K is the number of mutants killed and S is the number of surviving mutants. In reality, mutation testing is far more complicated than the formula implies. Some mutants are equivalent to the original program. Some are incompetent because they cannot be run meaningfully. Some time out. Some fail because of tool limitations rather than test weakness. In this dissertation, mutation score is therefore treated as a comparative metric over the mutation-evaluated subset, not as a complete measure of all generated tests.

Therefore, mutation scores should be interpreted comparatively, with attention to the mutation tool’s classification rules [11, 19, 20].

Failure-mode analysis is the last component. Aggregate metrics can hide differences between generation methods with identical final pass rates for different reasons. One route may struggle with imports, another with signatures, another with timeouts and another with assertion failures. Failure modes help explain why the pipeline repairs some failures and not others.

For this reason, this dissertation considers pass rates, coverage, mutation scores, runtime performance and failure modes collectively. The goal here is not to report a single headline figure. Rather, it is to examine whether repository-aware generation and bounded repair improve generated-test executability and measurable test quality, and where there are remaining limitations under favourable repository conditions.

2.9 CI/CD and Pull-Request-Facing Test Generation

Continuous integration is widely used in modern software projects to run tests automatically when code changes. This makes CI a natural setting for automated test

generation: if tests can be generated, validated and reported during a pull request, they become visible at a point where maintainers already review code [9].

A pull-request-facing workflow has requirements that differ from an offline experiment. It must be bounded and safe, account for the risks of running generated tests and pull-request code, and produce artefacts that maintainers can evaluate instead of silently modifying their repository. Literature on software bots indicates that automation can be valuable, but can also create noise, disruption and review overhead if its output is not carefully mediated [26].

LazyTest uses a GitHub Actions workflow as its repository-facing layer, while separate local pipelines configured for TheAlgorithms/Python provide the empirical basis for the main claims (see Chapter 3, Chapter 4, Figure 4.1 and Appendix B, Listing B.1). GitHub Actions demonstrates pull-request-facing integration; local pipelines provide controlled experimental data.

2.10 TheAlgorithms as a Controlled Case-Study Setting

The revised scope of this dissertation uses TheAlgorithms/Python as the main empirical case study. This decision is based on methodology, not on claiming that TheAlgorithms represents all Python repositories.

TheAlgorithms offers several advantages as an initial study for repository-aware test generation. It is open-source, Python-based and organised around numerous algorithmic components. The repository contains many algorithmic modules that are likely to have comparatively clear input-output behaviour, although this is not guaranteed for every target file. The repository also avoids much of the compiled-extension and build-system complexity present in projects such as NumPy or Pandas [25, 24]. These are broad suitability claims, not guarantees that every file in TheAlgorithms is simple or self-contained. Chapter 3 defines the discovery procedures and evaluated target corpus (see Chapter 3).

These favourable conditions make it possible to examine import handling, bounded context extraction and execution-aware repair without first relying on extensive framework configuration or compiled dependencies. They do not make the setting trivial: a large algorithm repository may still contain varied modules, optional dependencies, numerical code, edge cases, inconsistent documentation and import patterns that interfere with generated tests.

The limitation is explicit. Results from TheAlgorithms do not imply direct generalisation to arbitrary Python repositories. Dependency-heavy libraries such as Requests, data-science libraries such as Pandas or NumPy, web frameworks, projects with complex fixtures and projects requiring stronger environment management would likely require more robust dependency analysis, environment management and repository-context retrieval.

In this dissertation, TheAlgorithms serves to investigate a bounded research question: under favourable pure-Python conditions, how do measures of generated-test executability and measurable test quality vary when repository-aware context extraction, import handling and execution-aware repair are employed?

2.11 Summary and Research Gap

The literature points to a practical gap. SBST and Pynguin can search for inputs that improve structural coverage, while LLMs can use names, docstrings, examples and source code to generate plausible pytest tests. Execution feedback can repair some failures, and mutation testing can reveal where coverage does not correspond to fault-revealing strength.

However, target-file-only prompting is often too thin for repository-level Python testing, while whole-repository prompting is expensive, noisy and difficult to bound. Coverage-driven generation can produce tests that execute code without checking meaningful behaviour, and a repaired passing test may still be weak. API-based and locally served LLM backends may differ in capability and cost, but these differences only become meaningful when evaluated under a comparable pipeline and metric schema.

This dissertation addresses that gap through LazyTest: a CI-integrated, execution-aware pipeline evaluated through a controlled TheAlgorithms/Python case study. Its focus is repository-aware context extraction, import handling, bounded validation and repair, and evaluation through pass rate, coverage, mutation score, runtime and failure modes (see Chapter 5 and Appendix A, especially Tables A.5 and A.2). The claim is deliberately bounded: the project examines what is observed under favourable repository conditions, and which import, context and repair limitations remain.

Chapter 3

Methodology and Design

This chapter defines the methodology used to evaluate LazyTest under the revised scope of this dissertation. The evaluation is not a comprehensive benchmark across the entire Python ecosystem. It is a controlled case study focused on TheAlgorithms/Python, a favourable but bounded pure-Python repository through which to study repository-aware LLM-based unit-test generation.

This chapter distinguishes the pull-request-facing GitHub Actions workflow from the local empirical evaluation pipeline. It then defines the target corpus, generation conditions, repository-aware context construction, import handling, bounded validation and repair, and post-processing methods for coverage and mutation testing. The methodology supports the research questions presented in Chapter 1: executability, repair effectiveness, repository-aware generation, mutation-based fault-revealing ability, and remaining import/context limitations.

Generated tests are treated as regression-oriented or characterisation tests rather than as proof of semantic correctness. In the absence of an independent oracle, a generated test may preserve observed behaviour and may reveal regressions, but it cannot establish that the current implementation is correct [3]. This assumption affects both the repair loop and the interpretation of the evaluation metrics.

3.1 Controlled Case-Study Framing and System Boundary

The methodology follows a controlled case-study design. Case-study research is suitable where a contemporary software artefact is studied within its project context, provided that the case boundary, data sources and limitations are explicit [21]. Here, the case boundary is TheAlgorithms/Python, and the artefact assessed is LazyTest’s repository-aware, execution-aware test-generation pipeline.

TheAlgorithms/Python was chosen because it provides favourable conditions for an initial evaluation. Its repository structure and contribution guidance are broadly consistent with algorithm implementations that accept inputs, perform calculations or data manipulations, and return outputs, while encouraging conventions such as type hints, docstrings and doctests [25, 24]. These properties make it a suitable setting for studying whether LLM-generated pytest tests can be made executable and measurable

This choice is also a limitation. TheAlgorithms/Python is not treated as representative of all Python repositories, and the evaluation does not establish direct generalisation to dependency-heavy projects such as Requests, Pandas or NumPy. The aim is instead to examine what is observed when bounded repository-aware context and execution-aware repair are applied within this favourable case-study setting.

LazyTest consists of two related but distinct layers. The pull-request-facing GitHub Actions workflow demonstrates repository-facing orchestration, including ChatOps triggering, changed-file detection, generator selection, pytest validation, artefact handling and reporting (see Chapter 4, Figure 4.1, and Appendix B, Listing B.1). The main empirical claims in Chapter 5, however, come from a separate local pipeline configured for TheAlgorithms, which performs target discovery, generation, validation, bounded repair, logging, coverage recalculation and mutation analysis. This distinction keeps CI-integration evidence separate from the controlled evaluation data.

3.2 Target Discovery and Evaluation Sets

The local empirical pipeline discovers Python source files recursively from the checked-out TheAlgorithms repository. The discovery process ignores common non-source or generated directories, including `.git`, `__pycache__`, virtual-environment directories, `node_modules`, `build`, `dist` and `docs`. It also excludes files named `__init__.py` and files located under test directories. These are technical discovery filters rather than

Table 3.1: Evaluation sets and denominators used in the Chapter 5 analysis

Set	Definition	Role
Generation target set	The 1,144 discovered Python source files after technical filters	Denominator for pass-rate analysis
Accepted generated-test set	Targets for which generation finished with final status <code>ok</code>	Input for coverage recalculation
Coverage-evaluated set	Accepted tests rerun by the coverage stage	Basis for statement and branch coverage
Mutation-candidate set	Generated tests discoverable and mappable to a source file	Initial mutation-testing input
Primary mutation-score set	Mutation candidates with passing baseline pytest validation and completed Cosmic Ray sessions under the primary timeout	Basis for main mutation comparison
Extended-timeout retry set	Primary mutation timeouts rerun with a larger timeout budget	Diagnostic timeout-sensitivity analysis

post-hoc exclusions based on model performance.

Using this procedure before executing any models, the generation stage found 1,144 Python source files as target files (see Chapter 5, Table 5.2, and Appendix Tables A.1 and A.6). No additional semantic sampling rule was applied before generation (see Appendix B, Listing B.2, and Appendix Table A.1). Therefore, the resulting target set represents a broad within-repository sweep rather than a carefully curated set of simple examples, while still being bounded to TheAlgorithms.

When performing import normalisation for the local pipeline, missing `__init__.py` files may be created locally in the checkout so that modules can be imported consistently during generated-test execution. This is treated as an evaluation-environment accommodation rather than as a modification to the upstream repository.

The evaluation distinguishes multiple sets to avoid denominator confusion in Chapter 5.

Failed generation cases are included in pass-rate analyses. Coverage and mutation have narrower denominators because they require accepted generated tests to remain executable during later post-processing. Mutation score also requires successful completion of a mutation session.

3.3 Generation Conditions

This section defines the generation conditions evaluated in this study. Only routes with sufficiently complete pass-rate, coverage, mutation, runtime and failure-mode data are compared directly. Incomplete routes are treated as exploratory rather than as evidence of general model superiority (see Chapter 5 and Appendix Tables A.5 and A.16).

The main methodological contrast is between generic target-file generation and repository-aware LazyTest generation. The generic baseline receives the source code, source path and importable module name, but not LazyTest’s AST-extracted symbol lists, deterministic import preamble, import stripping, preloaded `target_module` interface or target-symbol constraints (see Appendix B, Listing B.5). This keeps the baseline realistic rather than deliberately weak, while withholding the repository-aware mechanisms being evaluated.

For the main RQ3 comparison, the generic and repository-aware conditions use the same outer validation and repair accounting: `pytest` validation, timeout policy, five-attempt budget and traceback feedback for later attempts. The comparison is therefore pipeline-level rather than a pure wording-only prompt ablation. Its intended contrast is between a conventional target-file pipeline and a repository-aware LazyTest pipeline with bounded symbol extraction and a deterministic import harness (see Chapter 4, Chapter 5, and Appendix B, Listings B.5–B.3).

The wider evaluation includes local open-weight routes, Pygwin where comparable metrics exist, and an API-based Gemini backend. These are interpreted as evaluated routes rather than as a universal model leaderboard, because prompts, serving environments, output limits, provider defaults and generation mechanisms differ. The Gemini condition used `gemini-3.1-flash-lite-preview` as a model substitution inside the same core repository-aware LazyTest pipeline, not as an autonomous coding-agent condition (see Appendix B, Listings B.9 and B.10, and Appendix Table A.3).

Prompt-repetition, base-prompt and mutation-focused variants are treated as run-level prompt-engineering configurations rather than defining features of LazyTest. Chapter 5 compares these variants only where complete quantitative data are available (see Table 5.5 and Appendix Table A.8).

3.4 Repository-Aware Context Construction and Import Handling

For each target file, the local repository-aware pipeline reads the source code and parses it using Python’s `ast` module. Top-level public functions and classes are extracted and included in the generation prompt to limit generation to real target symbols. Private definitions and functions named `main` are excluded. Where annotations can be parsed, they are added to the signature representation. The extracted list of symbols is limited to 50 items to prevent overly long prompts (see Appendix B, Listings B.2 and B.3).

This symbol extraction is deliberately conservative. It does not perform whole-repository semantic analysis, infer call relationships, or solve dynamic import behaviour. Instead, its purpose is to provide bounded local context: the source file, public symbols, signatures, available documentation and a controlled execution interface. The prompt instructs the model to use docstrings where available and to avoid testing `main()` or `if __name__ == "__main__"` blocks.

Import handling is central to the methodology because generated tests often fail before they reach the first assertion. Under the repository-aware condition, the model is instructed not to write imports and instead should call target functions through `target_module.<name>(...)`. After output extraction, model-generated imports are removed and a deterministic preamble is prepended to the test. This preamble imports required testing utilities, makes the repository available to the test process, loads the target file using a controlled module-loading mechanism, registers it under a stable `target_module` name, and exposes its public attributes to the generated test file (see Appendix B, Listings B.2 and B.3).

This approach does not claim to solve general Python import handling. It is a bounded method intended to reduce a specific class of generated-test failures in the controlled TheAlgorithms setting. It is most appropriate for target files that can be loaded directly and whose dependencies are available in the local environment. Failures caused by missing optional dependencies, import-time side effects, non-standard module initialisation or complex runtime requirements remain possible and are analysed as failure modes in Chapter 5.

The pull-request-facing GitHub Actions workflow uses a simpler import strategy for its direct LLM route. That workflow prompts the model to import the module under test by name and validates generated tests inside the checked-out pull-request

workspace. This difference reflects the different purposes of the two layers: the GitHub Actions workflow demonstrates pull-request-facing operation while the local pipeline evaluates the stricter repository-aware import harness.

3.5 Bounded Generate–Validate–Repair Loop

The local LLM-based generation pipeline uses a bounded Generate–Validate–Repair loop. Each target file receives up to five total attempts: one initial generation attempt and up to four repair attempts using pytest feedback from the previous failure (see Appendix B, Listing B.4).

After receiving a model response, the pipeline extracts Python code from mark-down code blocks or raw output containing test definitions. Visible reasoning or non-test explanatory material is removed where present. If no `def test...` function is found in the response, it is classified as invalid and the next attempt requests valid Python test code. After extraction, the pipeline removes model-generated imports, prepends the deterministic preamble, writes the generated test file and executes it with pytest. Validation runs use per-test and subprocess timeout values, which are reported in Chapter 4 alongside run configuration logs (see Appendix B, Listing B.4).

Generated suites are accepted when pytest exits successfully. If pytest fails, the pipeline captures stdout and stderr, truncates the error context where possible, extracts a short diagnostic note where available, and inserts the pytest failure output into the next repair prompt. The repair prompt requests the model to fix the root cause while preserving test strength. This is only a constraint at the prompt level rather than a formal guarantee; pass rate, coverage, mutation score and failure-mode metrics are therefore used in Chapter 5 to evaluate the success of repair rather than assuming semantically stronger tests were produced.

If a suite still fails after the five-attempt budget, it is recorded as failed and its corresponding generated test file is removed from the output directory so that invalid tests do not interfere with post-processing stages. Removing failed cases is an evaluation hygiene step rather than indicating how developers should treat failing cases. The failed case remains in the metrics CSV and failure logs, so it remains part of pass-rate and failure-mode analysis in Chapter 5. In a real repository environment, such a failure would require human interpretation: the failure may represent a generated-test failure, unsupported assumptions about environment usage, or in some cases ambiguities or defects in the target code.

The generation script records the target path, final status, elapsed time, generated-code length, number of generated test functions, number of attempts used and diagnostic notes (see Appendix Table A.1 and Appendix B, Listing B.4). These records support Chapter 5’s analyses of pass rate, repair improvement, runtime and failure modes.

The GitHub Actions workflow does not use this five-attempt loop in the same way. Direct LLM routes generate or refine tests, while Pynguin generates tests through its search-based route; both are validated in the pull-request-facing workspace. Thus the GitHub Actions workflow provides evidence for CI integration and PR-facing operation while the local TheAlgorithms pipeline is the basis for RQ2 on bounded execution-aware repair.

3.6 Pull-Request Workflow Methodology

The pull-request-facing GitHub Actions workflow provides implementation evidence for repository-facing orchestration design rather than as the primary experimental environment. It is included because the project also investigates PR-facing integration.

The workflow is triggered by comments on pull requests. A standalone ChatOps command selects the generator route and optional commit behaviour. Command parsing is limited to a controlled set of options to reduce the risk of unrestricted command-line execution from pull-request commenters. Prior to checking out pull-request code, a preflight stage retrieves pull-request metadata, checks the commenter’s authorisation and refuses forked pull requests by requiring the pull-request head repository to match the current repository. This trust boundary matters because the workflow may install dependencies, execute generated tests, and use write permissions or self-hosted runners.

After preflight, the workflow checks out the pull-request branch, identifies changed non-test Python files, creates an evaluation manifest, dispatches to the selected generator route, validates generated tests with pytest, and uploads generated tests, logs, metadata and coverage artefacts. It then reports results back to the pull request. Depending on configuration and validation policy, it may also commit generated tests and metadata back to the pull-request branch (see Chapter 4 and Appendix B, Listing B.1). Mutation testing is intentionally excluded from the PR workflow because it is too expensive for ordinary pull-request validation.

The treatment of failed generated tests differs between the local empirical pipeline

and the pull-request-facing workflow. In the local evaluation pipeline, failed final test files are removed from the generated-test directory to keep post-processing denominators clean. In the PR-facing workflow, failed tests are documented through logs, artefacts and pull-request comments, and commit behaviour is controlled by the selected commit mode. This reflects differences between a controlled evaluation pipeline and a developer-facing review workflow.

3.7 Structural Coverage Evaluation

Structural coverage is computed separately as a post-processing stage rather than embedded within the generation script (see Chapter 4 and Appendix B, Listing B.6). This separation is necessary because generation-time coverage fields are treated as diagnostics rather than as the authoritative source for the final evaluation.

Only tests with final status `ok` are considered for coverage evaluation. For each such test, the coverage analysis locates the corresponding generated test file, reruns `pytest` with branch coverage enabled and computes target-file statement coverage, target-file branch coverage and combined coverage. If the coverage rerun fails under strict counting, the target-file coverage is set to zero and the coverage-stage failure is documented separately (see Chapter 5, Table 5.2, Appendix Table A.11, and Appendix B, Listing B.6). This ensures that non-executable tests receive no credit for coverage.

The coverage analysis records target-file statement coverage percentage, target-file branch coverage percentage, combined coverage percentage, branch counts and coverage-stage status. Statement coverage indicates which executable statements were reached. Branch coverage indicates whether control-flow alternatives were exercised. Both are execution measures rather than evidence of semantic correctness [10].

3.8 Mutation-Score Evaluation

Mutation testing is performed offline using Cosmic Ray. It is not part of the pull-request-facing workflow because its computational cost is much higher than ordinary `pytest` validation. For each mutation task, the script maps a generated test file back to its source file and evaluates it in an isolated temporary copy of the repository (see Chapter 4 and Appendix B, Listing B.7). This prevents mutation testing from modifying the original checkout and reduces interference from repository-level configuration.

Before mutation testing, the generated test is executed against the unmodified

source file using `pytest`. Mutation analysis proceeds only if this baseline execution passes. If the baseline test fails or times out, no mutation score is reported for that test. This prevents invalid generated tests from receiving mutation-score credit.

The primary Cosmic Ray run uses a configured execution budget per target file, recorded in the mutation run configuration and reported with the Chapter 5 results (see Appendix B, Listing B.7, and Appendix Table A.16). Sessions that exceed this budget are recorded as timeout cases and excluded from the primary mean mutation score because their killed/survived denominator is incomplete (see Chapter 5, Table 5.2, and Appendix Tables A.2 and A.12). The primary metric is the viable mutation score:

$$MS_{\text{viable}} = \frac{K}{K + S} \times 100$$

where K is the number of killed mutants and S is the number of survived mutants. Incompetent mutants, timed-out mutants and other non-viable outcomes are logged separately and are not counted as killed. This avoids treating tool failures or incomplete mutation outcomes as evidence of fault-revealing behaviour.

The script also records a raw diagnostic mutation score:

$$MS_{\text{raw}} = \frac{K}{K + S + I + T + O} \times 100$$

where I , T and O denote incompetent, timed-out and other mutation outcomes respectively. The viable score is used as the main comparative measure because it focuses on mutants classified as viable by the mutation run. The raw score is retained only as a diagnostic value.

Timeout sensitivity is analysed separately. Files that time out during the primary mutation pass are rerun with a larger 900-second budget and lower parallelism (see Appendix B, Listing B.8, and Appendix Table A.16). These extended-timeout results are not merged into the primary comparison. They are reported as diagnostic evidence about computational cost and mutation-timeout sensitivity.

3.9 Metrics and Analysis

The evaluation integrates quantitative metrics and failure-mode analysis. A generated test suite is considered passing if the pipeline records final status `ok`. This status indicates executability under the validation harness, not semantic correctness of the target implementation.

The first metric is initial pass rate. This measures how many of the target files were able to produce a valid generated test suite on the first attempt, before using any repair attempt:

$$\text{Initial Pass Rate} = \frac{\text{Number of valid test suites on first attempt}}{\text{Total number of target files}} \times 100$$

Final pass rate uses the same total number of target files, but includes all test suites that were validated at any time during the five-attempt budget:

$$\text{Final Pass Rate} = \frac{\text{Number of validated test suites}}{\text{Total number of target files}} \times 100$$

Repair improvement is calculated using the difference between final pass rate and initial pass rate:

$$\text{Repair Improvement} = \text{Final Pass Rate} - \text{Initial Pass Rate}$$

While repair improvement provides useful information, it needs to be interpreted cautiously. If the final pass rate is greater than the initial pass rate, then more of the generated test suites have become executable after receiving feedback from pytest. However, this does not imply that the repaired tests became stronger. There could be instances where the later-stage repair addressed actual problems related to imports, signatures or assertions. On the other hand, the model could simply alter the generated content, choose a less difficult path through the code, or weaken an assertion. As a result, repair improvement is viewed as an executability and validation-success metric, and is evaluated alongside coverage, mutation score and failure-mode analysis.

Runtime is tracked separately for generation, validation, repair and mutation evaluation. Generation time, validation time and repair time come from the generation log. Mutation runtime comes from the mutation script (see Chapter 5, Table 5.3, and Appendix Table A.10). Because mutation testing occurs offline as a separate evaluation phase, these runtimes are not aggregated into a single overall runtime. Additionally, runtime comparisons require consideration of differing execution environments and serving overheads due to different backend configurations, for example Pygwin, local open-weight model runs, quantised model runs and API-based backends.

Failure-mode analysis provides additional insight into why certain aggregate metrics occur. During the generation phase, the pipeline collects data about categories of

failures such as invalid model output format errors, import errors, module-not-found errors, wrong signature errors, assertion failures and timeout errors. The mutation phase collects additional categories of failures, such as baseline failures, syntax errors, missing dependencies, Cosmic Ray initialisation failures, execution timeout errors and database parsing errors. Chapter 5 evaluates whether failures in these categories are primarily caused by differences in model output formats, repository context, import handling, behaviour of target code, weak assertions, limitations in repair or constraints of mutation tools (see Chapter 5, Table 5.7, and Appendix Tables A.13 and A.14).

3.10 Methodological Limitations

There are several methodological limitations. First, the evaluation centres on TheAlgorithms/Python and the 1,144 source files selected by technical filters. This supports a controlled within-repository study, but not direct generalisation to dependency-rich projects, web applications or the wider Python ecosystem.

Second, the repository-aware condition combines prompt constraints, AST-based symbol extraction, deterministic import handling, output sanitisation and bounded repair. The comparison with the generic target-file baseline is therefore pipeline-wide, not a pure prompt ablation.

Third, backends are compared as evaluated routes rather than as pure model architectures. Qwen, Devstral, Gemini and Pynguin differ in prompts, serving or API configuration, output limits, cost constraints and generation mechanisms, so backend differences are interpreted cautiously.

Finally, generated tests are not correctness proofs. Without an independent oracle, generated assertions may preserve current behaviour even if that behaviour is incomplete or flawed. Mutation scores provide comparative fault-revealing evidence, but remain tool-dependent and affected by equivalent mutants, incompetent mutants, timeouts and operator choice [11, 19, 20]. Repair improvement indicates additional executability, not necessarily stronger assertions. The GitHub Actions workflow and local empirical pipeline also serve different objectives, so their results are not interchangeable.

3.11 Chapter Summary

This chapter defined the methodology for evaluating LazyTest under the revised controlled case-study framework. It separated the pull-request-facing GitHub Actions workflow from the local empirical pipeline and used 1,144 technically discovered TheAlgorithms/Python files rather than a hand-picked subset.

The main RQ3 comparison is between generic target-file generation and repository-aware LazyTest generation under the same outer validation and repair process. Other routes, including Pynguin, local open-weight models and Gemini, are treated as specified pipeline routes where complete metrics are available, not as a general model leaderboard.

Coverage and mutation are separate post-processing phases with narrower denominators than generation pass rate. Primary mutation results use completed sessions within the configured budget, while 900-second timeout retries remain diagnostic. These decisions support Chapter 5's analysis of pass rate, repair improvement, structural coverage, viable mutation score, runtime overhead and failure modes.

Chapter 4

Implementation

This chapter describes how LazyTest was implemented under the revised scope of the dissertation. Chapter 3 established three points that shape the implementation: the dissertation is a controlled TheAlgorithms/Python case study rather than a broad generalisation claim; the system has a pull-request-facing workflow layer and a separate local empirical evaluation layer; and the main evaluation uses executability, bounded repair, coverage, mutation score, runtime and failure modes.

The implementation consists of two related but distinct layers. The first is a GitHub Actions workflow that exposes LazyTest as a pull-request-facing ChatOps tool for requesting, validating and reporting generated tests. The second is a local TheAlgorithms evaluation pipeline used to produce the controlled data analysed in Chapter 5. This local layer contains the mechanisms evaluated in this dissertation: repository-aware context extraction, deterministic import handling, bounded pytest repair and offline post-processing for coverage and mutation analysis. The workflow is therefore orchestration evidence, while the local scripts provide the controlled evaluation artefacts. Figure 4.1 summarises this separation.

4.1 Pull-Request-Facing GitHub Actions Workflow

The GitHub Actions workflow implements LazyTest’s pull-request-facing layer. It is triggered from issue comments so that a maintainer or collaborator can request generated tests during review rather than running the local research scripts manually. The workflow is implemented in `.github/workflows/lazytest-generator.yml`, with representative command-parsing and preflight excerpts in Appendix B, Listing B.1. It addresses the CI/CD integration objective, but it is not the controlled case-study

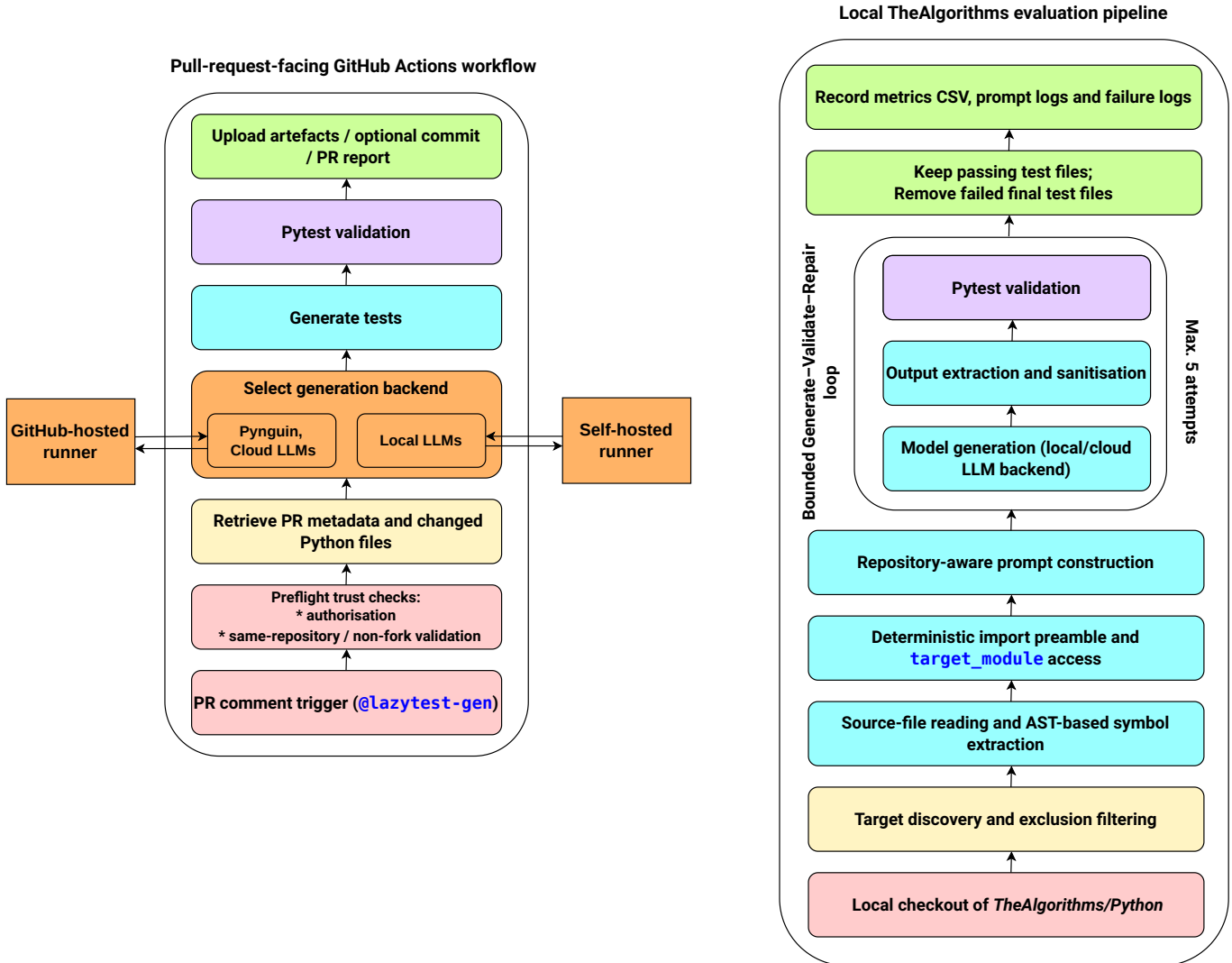


Figure 4.1: LazyTest implementation architecture, distinguishing the pull-request-facing GitHub Actions workflow from the local TheAlgorithms evaluation pipeline used to produce the controlled Chapter 5 artefacts. The right-hand pipeline contains the repository-aware context extraction, deterministic import handling, bounded pytest repair and metric-collection stages evaluated in this dissertation.

Table 4.1: Implemented LazyTest components and evidence sources

Component	Implementation role	Main evidence
GitHub Actions workflow	Pull-request-facing ChatOps generation, validation, reporting and artefact handling	<code>.github/workflows/lazy-test-generator.yml</code> ; Appendix B, Listing B.1
Repository-aware generator	Main local LazyTest generation pipeline for TheAlgorithms	Main evaluated <code>generate_tests_*.py</code> scripts; Appendix B, Listings B.2–B.4
Gemini API backend	API-based model substitution inside the repository-aware LazyTest pipeline	Gemini-specific <code>generate_tests_*.py</code> script; Appendix B, Listing B.9
Generic target-file baseline	Simpler LLM baseline without the repository-aware import harness	<code>generate_baseline.py</code> ; Appendix B, Listing B.5
Structural coverage recalculation	Separate statement and branch coverage post-processing	<code>recalc_cov.py</code> ; Appendix B, Listing B.6
Mutation evaluation	Offline Cosmic Ray mutation-score computation	<code>mutation_score.py</code> ; Appendix B, Listing B.7
Mutation timeout retry	Diagnostic rerun of primary mutation timeouts	<code>retry_mutation_timeouts.py</code> ; Appendix B, Listing B.8

pipeline used for the Chapter 5 evaluation.

At a high level, the workflow parses the triggering command, performs preflight checks, resolves the pull-request context, identifies changed Python files, dispatches the selected generator, validates the resulting tests, uploads artefacts and reports back to the pull request. The preflight stage provides a trust boundary because the workflow may install dependencies, execute generated tests, and in some configurations write files back to the pull-request branch. It checks the triggering user’s repository role and refuses forked pull requests before checkout, reducing the risk that untrusted pull-request code is executed with repository write permissions or self-hosted runner access. This is not a complete security solution, but it is an explicit implementation-level safeguard.

The `issue_comment` payload does not contain all information needed for target selection, so the workflow retrieves branch metadata, computes the base-to-head diff and filters candidate files to exclude obvious tests and other non-target files. It supports

search-based generation, direct LLM generation, local-LLM generation and refinement-style routes. Under the revised framing, this backend variety demonstrates orchestration flexibility, whereas the main quantitative claims are based on the local TheAlgorithms pipeline. Its direct LLM route uses a simpler module-name import strategy and does not implement the full AST extraction, deterministic preamble and five-attempt local repair loop used in the TheAlgorithms evaluation pipeline.

Once generation completes, the workflow validates generated tests with `pytest`, uploads generated tests, logs and metadata as artefacts, optionally commits generated tests back to the pull-request branch, and posts a structured report. The workflow therefore embeds LazyTest in a review process, but remains intentionally lighter than the local evaluation pipeline described next.

4.2 Local Repository-Aware Generation Pipeline

The local repository-aware pipeline implements the main mechanisms evaluated in this dissertation. It processes source files from a local TheAlgorithms checkout, generates candidate `pytest` suites, validates them immediately, repairs failures within a bounded retry budget, and records per-file outcomes. The evaluated Qwen, Devstral and Gemini routes are implemented in model-specific `generate_tests_*.py` scripts which share the same generation, validation and repair structure while differing mainly in model endpoint, prompt template and run configuration. The Gemini route used `gemini-3.1-flash-lite-preview` as an API backend inside the same repository-aware generation, validation and repair structure, rather than as an autonomous coding-agent condition. The AST extraction, deterministic preamble and repair loop are shown in Appendix B, Listings B.2–B.4.

Target discovery recursively finds Python files while excluding common non-target directories such as caches, virtual environments, build artefacts and test directories. Files such as `__init__.py` are also excluded. In the controlled case-study run used for Chapter 5, this procedure produced 1,144 Python source files, as recorded in the generation metrics and discovery logs (see Chapter 5 and Appendix A, Tables A.1 and A.6).

For each source file, the pipeline reads the source and derives bounded repository-aware context. It uses Python’s `ast` module to extract top-level public functions and classes, including signatures where available. This is not full semantic analysis; its purpose is to ground the prompt in real target symbols and reduce the opportunity

Table 4.2: Main steps in the deterministic import preamble

Preamble step	Purpose
Add the repository root to <code>sys.path</code>	Make local modules discoverable during execution
Create package modules where required	Support nested repository module paths
Remove stale module entries where required	Reduce collisions from previous imports
Load the target file via <code>importlib.util</code>	Avoid reliance on model-generated imports
Register the loaded module as <code>target_module</code>	Provide a stable access interface for generated tests
Expose relevant public attributes	Support consistent generated-test access

for unsupported target-symbol use. The prompt combines the source code, extracted symbols and execution instructions. Some runs vary wording, such as adding mutation-aware goals or prompt repetition, but these are run-level configurations rather than mandatory parts of every route.

The pipeline enforces a controlled target access pattern. Generated tests are instructed to call functions through `target_module.<name>(...)`, while the script strips model-generated imports and prepends a deterministic preamble. The preamble adds the repository root to `sys.path`, creates package modules where needed, clears stale module entries, loads the target file using `importlib.util`, registers it as `target_module`, and exposes relevant public attributes. Table 4.2 summarises these steps.

This preamble is a bounded engineering mechanism for the TheAlgorithms evaluation environment, not a complete solution to Python import handling. Import-time side effects, missing dependencies, unusual packaging structures or internal assumptions may still cause failures, which are recorded in the failure-mode analysis. The pipeline also sanitises model output by extracting usable Python test code from markdown fences or explanatory text. If no valid `def test_...` functions are found, the attempt is treated as invalid and a repair-style prompt requests a proper pytest file.

Each candidate test is validated immediately with pytest. The script writes a temporary file, adjusts `PYTHONPATH`, and runs pytest on the single generated file with timeouts and compact traceback reporting. Passing files are accepted and kept; failing files produce traceback feedback for the next repair prompt. The loop is bounded to five

total attempts, as abstracted in Algorithm 1. Chapter 5 evaluates the resulting artefacts rather than assuming every repair improves semantic test strength.

Algorithm 1 Repository-aware Generate–Validate–Repair loop

```

1: for all source_file in discovered_targets do
2:   source_code ← read(source_file)
3:   symbols ← extract_symbols_with_ast(source_code)
4:   prompt ← build_repository_aware_prompt(source_code, symbols)
5:   for attempt = 1 to 5 do
6:     raw_output ← call_model(prompt)
7:     test_code ← extract_and_sanitise(raw_output)
8:     if no usable pytest tests are found then
9:       prompt ← build_invalid_output_prompt(source_code, raw_output)
10:      continue
11:    end if
12:    final_test ← deterministic_preamble(source_file) + test_code
13:    result ← run_pytest(final_test)
14:    if result.passed then
15:      keep_test_file()
16:      record_status_ok()
17:      break
18:    end if
19:    prompt ← build_repair_prompt(source_code, test_code, result.traceback)
20:  end for
21:  if no attempt passes then
22:    remove_final_test_file()
23:    record_failure()
24:  end if
25: end for

```

If all attempts fail, the final test file is removed so that invalid tests are not included in later coverage or mutation stages. The failure is still recorded in the metrics and logs, so it remains part of the pass-rate and failure-mode analysis. Removal is therefore an implementation-hygiene step, not an exclusion from the evaluation denominator.

4.3 Generic Target-File Baseline

A separate script, `generate_baseline.py`, implements the generic target-file baseline used for comparison with the repository-aware pipeline. It provides a simpler LLM route without LazyTest’s repository-aware mechanisms, rather than an artificially weak baseline.

The baseline prompt gives the model the source code, the source path relative to the repository root, and an importable module name. It asks for raw `pytest` code, discourages reimplementation, and imports the target through `importlib.import_module(...)`. The baseline adjusts `PYTHONPATH`, but deliberately omits AST-based symbol extraction, the deterministic import preamble, import stripping, and the preloaded `target_module` interface.

The baseline is not a no-repair condition. It uses the same bounded repair structure as the main pipeline: tests are validated with `pytest`, failures are fed back into the next prompt, and up to five total attempts are allowed (see Appendix B, Listing B.5). The Chapter 5 comparison is therefore a pipeline-level comparison between a generic target-file LLM workflow and a repository-aware workflow that adds context extraction and deterministic import handling. The baseline CSV follows the same core schema as the repository-aware generator, although the authoritative coverage values are produced by the separate post-processing stage.

4.4 Structural Coverage Recalculation

Structural coverage is evaluated in a separate post-processing step, implemented in `recalc_cov.py`, rather than taken directly from generation-time scripts. This provides a more consistent basis for comparing accepted generated tests across run folders.

For each run, the script reads the original metrics CSV, preserves the rows, and appends coverage-related columns. Rows whose generation status is not `ok` receive no coverage credit because no accepted final test exists. Accepted tests are rerun individually under `pytest` with `pytest-cov`, branch coverage, JSON output and configured timeouts (see Appendix B, Listing B.6). The target file is matched using its repository-relative path, which avoids basename ambiguity.

The script extracts statement coverage, branch coverage, combined coverage and branch counts. It also records coverage-stage diagnostics such as missing test file, missing coverage JSON, target file missing from coverage output, `pytest` failure, timeout and evaluation exception. Under the strict policy, coverage is set to zero if `pytest` fails during the coverage-evaluation rerun. Chapter 5 therefore distinguishes generation success from successful structural coverage recomputation.

4.5 Mutation-Score Evaluation

Mutation testing is implemented as a separate offline stage using Cosmic Ray, because it is substantially more expensive than ordinary pytest validation. The implementation is contained in `mutation_score.py`, with the baseline validation, temporary-copy setup and Cosmic Ray invocation shown in Appendix B, Listing B.7.

For each run folder, the script discovers generated `test_*.py` files, maps each test back to its source file, and executes each task in an isolated temporary copy of the repository. This reduces interference between parallel tasks and avoids modifying the original checkout. The generated test is copied into the temporary context, paths are rewritten where necessary, and unrelated repository pytest configuration may be removed.

Before invoking Cosmic Ray, the script validates the generated test against the unmutated source file. Cases that fail or time out at this stage are not mutation-tested, because invalid tests should not receive mutation-score credit. If validation succeeds, the script writes a Cosmic Ray configuration, runs `cosmic-ray init` and `cosmic-ray exec` under explicit timeout budgets, and parses the resulting SQLite database. The parser searches for an outcome-like column and counts killed, survived, incompetent and timed-out mutants.

The main reported mutation metric is the viable mutation score:

$$MS_{\text{viable}} = \frac{K}{K+S} \times 100$$

where K is the number of killed mutants and S is the number of survived mutants. Incompetent, timed-out and other outcomes are logged separately rather than counted as killed. The script also records a diagnostic raw score:

$$MS_{\text{raw}} = \frac{K}{K+S+I+T+O} \times 100$$

where I , T and O denote incompetent, timed-out and other outcomes. The viable score is the main comparative metric because it focuses on mutants successfully processed and classified as viable. Rows with no viable killed or survived mutants receive a separate `no_viable_mutants` status rather than being included in the viable-score mean.

The mutation script records statuses including baseline failure, baseline timeout, initialisation timeout, execution timeout, database parse failure, no mutants, no viable

Table 4.3: Main implementation artefacts and their analytical role

Artefact	Produced by	Role in later analysis
Generation metrics CSV	Local generation scripts	Pass rate, attempts, runtime and notes
Prompt debug log	Local generation scripts	Prompt and output inspection
Failure log	Local generation scripts	Failure-mode diagnosis
Coverage CSV	Structural coverage script	Statement and branch coverage analysis
Mutation CSV	Mutation script	Mutation-score analysis
Retry mutation CSV and summary JSON	Timeout retry script	Timeout sensitivity analysis
Workflow metadata and artefacts	GitHub Actions workflow	PR-run inspectability and reproducibility

mutants and successful mutation processing. These statuses matter because mutation analysis uses a narrower denominator than raw generation or coverage analysis. A separate retry script, `retry_mutation_timeouts.py`, reruns primary mutation timeouts with a larger execution budget. In the configured retry analysis, this timeout is 900 seconds, as recorded in the retry logs and summary JSON (see Appendix B, Listing B.8, and Appendix A, Table A.16). Retry outputs are written separately, keeping them as diagnostic timeout-sensitivity evidence rather than a redefinition of the primary mutation denominator.

4.6 Artefacts, Reproducibility and Implementation Constraints

The implementation produces several artefacts that support later analysis and reproducibility.

The generation scripts write per-run CSV files recording source file, status, generation time, number of tests, attempt count and notes. The coverage stage adds structural coverage data, while the mutation stage writes per-run and combined mutation outputs. The workflow layer records metadata about PR-facing runs. These artefacts provide

the raw basis for the Chapter 5 tables and denominator checks, and their model/run-folder organisation allows aggregate values to be traced back to generated tests and logs.

The implementation has clear boundaries. These scripts are research artefacts for a controlled case study, not a fully general production framework. Paths, model endpoints, run folders and some execution settings are managed through script configuration or command-line arguments rather than a single polished user-facing system. The repository-aware import harness is also bounded: it is intended to reduce import fragility in the TheAlgorithms environment, but does not solve Python package loading for all repository structures. Similarly, AST-based symbol extraction constrains the model to real top-level targets without providing behavioural specifications or solving the oracle problem.

This oracle limitation affects how failed generated tests should be interpreted. In the local evaluation, the current TheAlgorithms implementation is treated as the behavioural reference so that tests can be validated consistently. In a real pull-request setting, a persistent failure may indicate an invalid generated test, an import or fixture problem, under-specified behaviour, or a genuine defect. LazyTest therefore produces candidate tests and diagnostic evidence for review, rather than replacing developer judgement.

Overall, the implementation provides the mechanisms required by the revised methodology: a PR-facing workflow layer, a repository-aware local generation pipeline, a generic baseline, a Gemini API backend inside the same pipeline, structural coverage recalculation and offline mutation-score evaluation. Chapter 5 uses the artefacts produced by these components to evaluate LazyTest in terms of executability, bounded repair improvement, coverage, mutation score, runtime and failure modes.

Chapter 5

Evaluation and Results

This chapter reports the empirical evaluation of LazyTest under the controlled TheAlgorithms/Python case-study design defined in Chapter 3. It does not present a broad repository benchmark or model leaderboard. Instead, it evaluates the implemented generation conditions on the fixed TheAlgorithms target set, focusing on executability, bounded repair, structural coverage, mutation score, runtime overhead and failure modes.

The evaluation uses 1,144 Python source files selected by the technical discovery filters described in Chapter 3; see Appendix Tables A.1 and A.6. Pass rates use this full target set. Coverage is reported both over accepted tests rerun by the coverage stage and, where stated, in target-normalised form; viable mutation score is computed only for tests whose mutation sessions produced killed or survived viable mutants. This avoids treating accepted-test quality metrics as if they used the same denominator as pass rate.

Percentages are reported to two decimal places. “Repair improvement” is final pass rate minus initial pass rate. Because the raw metrics did not store a separate first-attempt status column, initial pass rate is inferred from final ok cases with `attempts <= 1`; it is therefore a first-attempt proxy rather than a separately logged status (Appendix Tables A.2 and A.16).

The tables use shortened labels. “Qwen 27B” and “Qwen 9B” refer to `Qwen/Qwen3.5-27B` and `Qwen/Qwen3.5-9B`; AWQ labels refer to the corresponding `cyankiwi` 4-bit routes. “Gemini 3.1 Flash Lite” refers to `gemini-3.1-flash-lite-preview`, while “Devstral AWQ” and “Qwen3-Coder-Next AWQ” refer to the named `cyankiwi` routes. The AWQ analysis is route-level evidence, not a controlled quantisation study.

5.1 Main Pipeline Comparison

The main comparison addresses RQ3 at pipeline level. The generic baseline receives the source file, path and importable module name, but not LazyTest’s AST-derived symbol list, deterministic preamble, import stripping or controlled `target_module` access. Both conditions still use bounded pytest validation and repair, so the contrast is not repair versus no repair.

Table 5.1 compares the three matched settings where both generic and repository-aware runs are available.

Table 5.1: Main comparison between generic target-file generation and repository-aware LazyTest generation on the 1,144-file TheAlgorithms target set

Route	Pipeline	Init. pass %	Final pass %	Repair pp	Eval. cov. %	Target-norm. cov. %	Mutation %	Mut. <i>n</i>
Qwen 27B	Repository-aware	72.90	95.10	22.20	80.74	64.86	70.47	1080
Qwen 27B	Generic baseline	33.74	91.70	57.95	81.91	63.01	73.86	1040
Qwen 9B	Repository-aware	48.43	77.19	28.76	76.87	49.86	68.63	878
Qwen 9B	Generic baseline	29.81	71.15	41.35	79.75	47.12	72.88	810
Qwen 9B AWQ	Repository-aware	44.93	75.87	30.94	76.53	48.90	65.86	864
Qwen 9B AWQ	Generic baseline	31.03	66.52	35.49	79.28	44.00	70.92	758

Notes: “Eval. cov.” is mean combined coverage over accepted coverage-evaluated tests; “target-norm. cov.” treats missing or failed coverage as zero over the full target set. Mutation is mean viable mutation score. Source: `main_pipeline_comparison.csv`; see Appendix Table A.4.

Separate statement and branch coverage values are reported in Appendix Table A.7.

Table 5.2: Evaluation denominators for the main matched routes

Route	Pipeline	Target <i>n</i>	Final ok <i>n</i>	Cov. eval. <i>n</i>	Cov. failed <i>n</i>	Mut. cand. <i>n</i>	Mut. viable <i>n</i>
Qwen 27B	Repository-aware	1144	1088	1088	169	1088	1080
Qwen 27B	Generic baseline	1144	1049	1049	169	1049	1040
Qwen 9B	Repository-aware	1144	883	883	141	883	878
Qwen 9B	Generic baseline	1144	814	814	138	814	810
Qwen 9B AWQ	Repository-aware	1144	868	868	137	868	864
Qwen 9B AWQ	Generic baseline	1144	761	761	126	761	758

Notes: “Final ok” counts validated generated suites; coverage and mutation columns record how many accepted tests reached later post-processing stages. Source: `condition_summary.csv` and `all_file_results.csv`; see Appendix Table A.6.

Table 5.2 shows that repository-aware routes usually retain more targets through

validation, coverage and mutation, even when surviving generic-baseline tests have competitive accepted-test metrics.

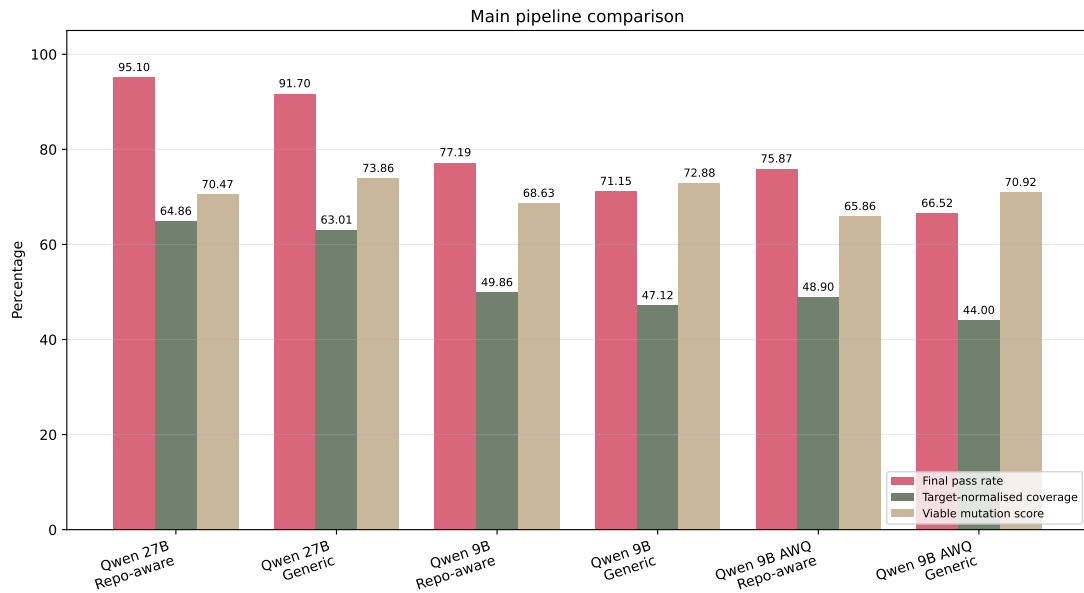


Figure 5.1: Final pass rate, target-normalised combined coverage and viable mutation score for the main matched routes. Source: `main_pipeline_comparison.csv`; see Appendix Table A.4.

The clearest result is higher executability in every matched main comparison. Final pass rate rises from 91.70% to 95.10% for Qwen 27B, from 71.15% to 77.19% for Qwen 9B, and from 66.52% to 75.87% for Qwen 9B AWQ. Initial pass-rate gaps are larger, especially for Qwen 27B, where repository-aware generation starts at 72.90% compared with 33.74% for the generic baseline. This supports the pipeline-level conclusion that the combined repository-aware condition is associated with higher first-pass executability; it does not isolate the effect of any individual mechanism.

The accepted-test quality metrics are more mixed. Generic baselines sometimes have slightly higher evaluated coverage or viable mutation score among surviving tests, such as the Qwen 27B baseline’s 73.86% mutation score compared with 70.47% for the repository-aware route. However, target-normalised combined coverage is higher for the repository-aware route in all three matched comparisons because it retains more files. The main advantage of repository-aware LazyTest is therefore full-target-set executability, not dominance on every accepted-test quality metric.

5.2 Repair Effectiveness and Executability

RQ2 asks whether bounded execution-aware repair increases the number of passing generated suites. It does across all main conditions, with larger gains where initial generation is weaker. In the main comparison, repair improvement ranges from 22.20 percentage points for the Qwen 27B repository-aware route to 57.95 percentage points for its generic baseline. This shows that traceback-based repair improves executability, but not necessarily semantic test strength.

Table 5.3: Recorded generation time and repair-attempt summary for the main matched routes

Route	Pipeline	Recorded gen. time h	Median per-file time s	Mean attempts	Mean attempts ok
Qwen 27B	Repository-aware	22.46	41.31	1.51	1.34
Qwen 27B	Generic baseline	96.36	181.84	2.24	2.00
Qwen 9B	Repository-aware	122.62	183.36	2.43	1.67
Qwen 9B	Generic baseline	123.06	246.11	2.90	2.05
Qwen 9B AWQ	Repository-aware	145.45	222.08	2.55	1.77
Qwen 9B AWQ	Generic baseline	196.98	452.51	2.99	1.98

Notes: Recorded generation time is summed per-file time from the local scripts, not hardware-normalised wall-clock time. “Mean attempts ok” is computed only over validated targets. Source: `condition_summary.csv`; see Appendix Table A.10.

The runtime data are practical execution evidence rather than a controlled speed benchmark. They nevertheless show that weaker initial generation usually produces more repair work: generic baselines require more attempts than the corresponding repository-aware routes. As argued in Chapter 3, a repaired test is more executable because it passes validation, but it is not automatically a better test. Repair is therefore interpreted alongside coverage, mutation score and failure modes.

5.3 Coverage and Mutation Score

RQ4 asks whether generated tests show fault-revealing behaviour beyond structural coverage. Across the main LLM conditions, evaluated combined coverage ranges from 76.53% to 81.91%, while viable mutation score ranges from 65.86% to 73.86%. These scores suggest that accepted tests often detect viable mutants, but coverage and mutation do not rank routes identically. Structural coverage shows what code is executed; mutation score gives stronger comparative evidence that tests detect behavioural

changes, although it remains limited by equivalent mutants, timeouts and the narrower mutation-evaluated denominator. This means mutation score is used as comparative evidence of fault-revealing behaviour, not as a correctness measure.

5.4 Secondary Backend and Prompt Results

Secondary routes are not the central RQ3 comparison, but they show how LazyTest behaves across selected backends and prompts. Table 5.4 reports representative conditions; additional statement and branch coverage values are in Appendix Table A.7.

Table 5.4: Selected secondary routes and prompt variants

Route	Prompt / condition	Final pass %	Eval. cov. %	Mutation %	Mut. <i>n</i>
Pynguin	Search-based generation	60.58	71.52	40.65	660
Gemini 3.1 Flash Lite	Repository-aware API backend	90.65	81.11	72.22	1028
Qwen 27B	Base prompt, repeated	96.68	80.93	70.60	1099
Qwen 27B	Mutation-aware softened, repeated	93.27	81.96	74.23	1057
Devstral AWQ	Base prompt, repeated	78.41	80.75	71.35	892
Qwen3-Coder-Next AWQ	Base prompt, repeated	89.86	79.76	70.09	1021

Notes: These are route-level case-study results, not a model leaderboard. Pynguin “final pass” uses the same validation/evaluation acceptance criteria, but not the LLM repair loop. Source: `secondary_backend_summary.csv`; see Appendix Table A.5.

Pynguin provides a search-based point of comparison: 60.58% final pass rate, 71.52% evaluated combined coverage and 40.65% viable mutation score. This is below the selected LLM-based LazyTest routes here, but should not be read as a general Pynguin result. The Gemini route used `gemini-3.1-flash-lite-preview` inside the same repository-aware pipeline and reached 90.65% final pass rate, 81.11% coverage and 72.22% mutation score, showing that, in this implementation, an API backend was substituted while preserving the same evaluation structure. Prompt changes also alter the executability–mutation trade-off, which is analysed next.

5.5 Effect of Mutation-Aware Prompting

The Qwen/Qwen3.5–27B route provides the clearest prompt-variant comparison. Table 5.5 contrasts the base prompt with strict and softened mutation-aware prompts; the

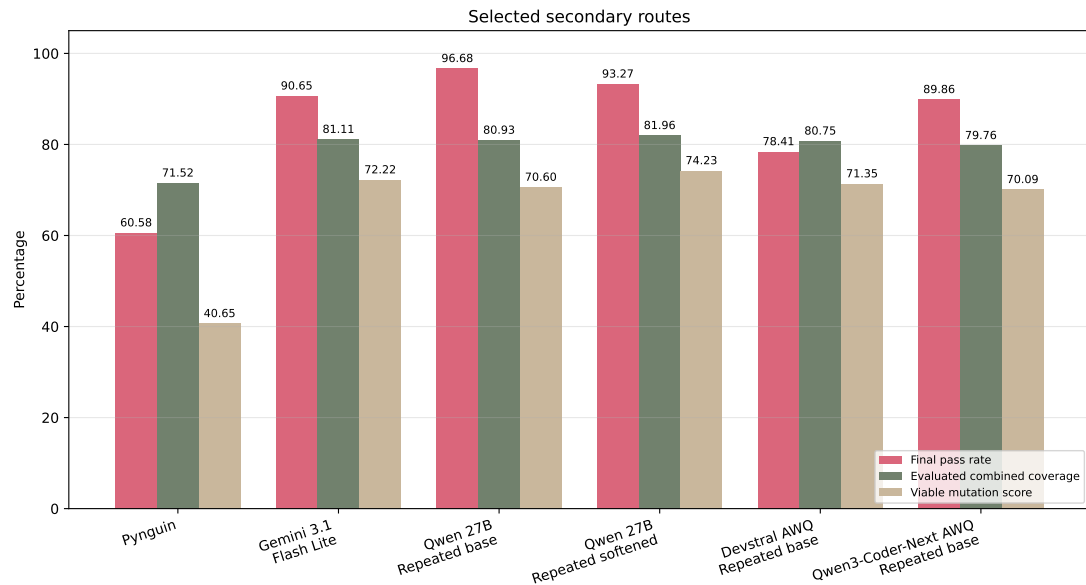


Figure 5.2: Final pass rate, evaluated combined coverage and viable mutation score for selected secondary routes. Source: `secondary_backend_summary.csv`; see Appendix Table A.5.

strict version included explicit 100% line-coverage wording.

The base prompt gives the highest final pass rates: the repeated base run reaches 96.68%, the highest route observed. Mutation-aware prompts improve viable mutation score by roughly three to four percentage points, with the repeated softened version reaching 74.23% compared with 70.60% for repeated base. This indicates a trade-off: mutation-aware prompting was associated with higher viable mutation score among accepted tests, but lower executability. The softened wording slightly outperforms the strict version in the repeated setting, suggesting, for this route, that absolute coverage targets may have encouraged more brittle outputs. This remains a route-specific observation, not a general prompt-engineering law.

5.6 Local and Quantised Route Analysis

Local and quantised routes are included as secondary practical evidence, not as a formal quantisation study. Table 5.6 summarises matched Qwen full and AWQ conditions using averages over matched runs rather than pooled target-level results.

For Qwen 27B, the AWQ route remains close to the full route: final pass rate is

Table 5.5: Qwen 27B prompt variants under the repository-aware LazyTest pipeline

Prompt condition	Repeated	Final pass %	Eval. cov. %	Mutation %
Base	No	95.10	80.74	70.47
Base	Yes	96.68	80.93	70.60
Mutation-aware strict	No	86.98	81.74	73.92
Mutation-aware strict	Yes	92.22	82.00	74.01
Mutation-aware softened	No	89.51	81.51	73.30
Mutation-aware softened	Yes	93.27	81.96	74.23

Notes: “Strict” uses the earlier explicit 100% coverage target; “softened” uses revised mutation-aware wording. Source: `secondary_backend_summary.csv`; see Appendix Table A.8.

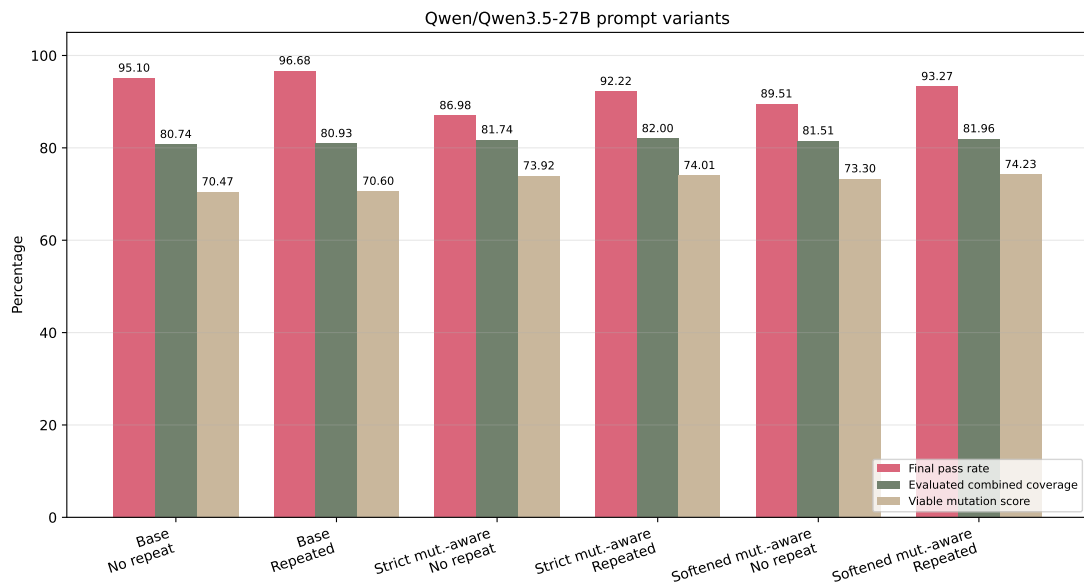


Figure 5.3: Final pass rate, evaluated combined coverage and viable mutation score across Qwen/Qwen3.5-27B prompt variants. Source: `secondary_backend_summary.csv`; see Appendix Table A.8.

Table 5.6: Matched comparison of full and AWQ-quantised Qwen routes

Matched route pair	Runs	Full pass %	AWQ pass %	Full cov. %	AWQ cov. %	Full mut. / AWQ mut. %
Qwen 27B vs Qwen 27B AWQ	5	92.10	90.42	81.39	81.30	72.46 / 72.10
Qwen 9B vs Qwen 9B AWQ	5	66.38	62.24	78.55	78.28	71.17 / 69.89

Notes: Values are matched-condition averages used only as secondary route-level evidence. Source: `condition_summary.csv`; see Appendix Table A.9.

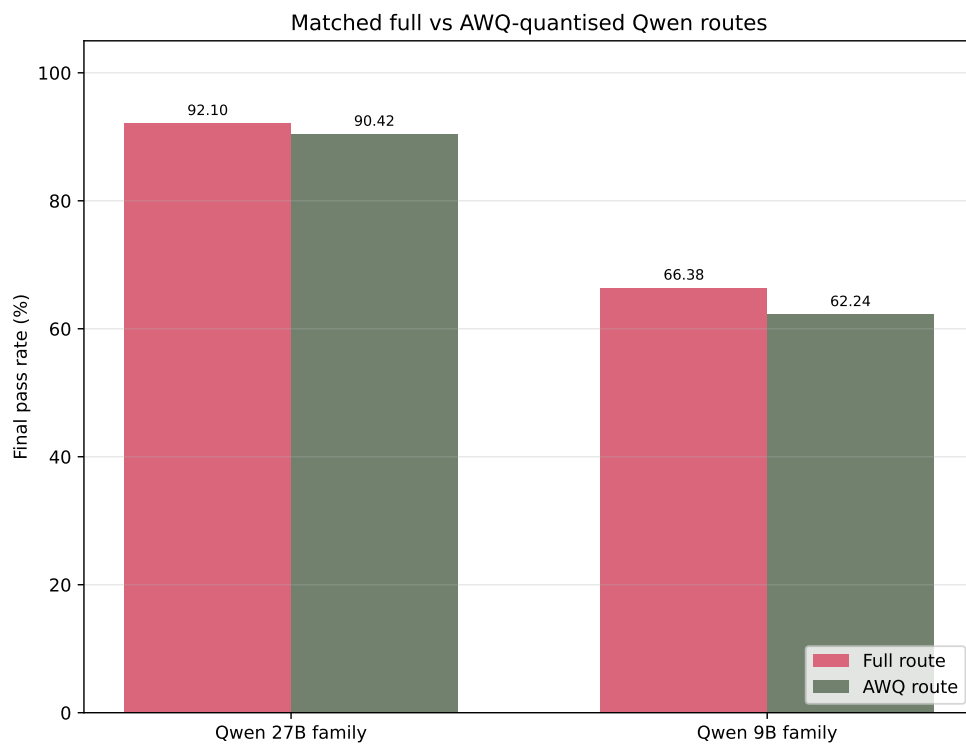


Figure 5.4: Average final pass rate for matched full and AWQ-quantised Qwen route pairs. Source: `condition_summary.csv`; see Appendix Table A.9.

lower by 1.68 percentage points, while coverage and mutation score are nearly unchanged. For Qwen 9B, AWQ is weaker on pass rate by 4.14 percentage points, with similar coverage and slightly lower mutation score. These results show that the evaluated AWQ routes remained capable of producing accepted tests at substantial rates in LazyTest, but they do not establish a general relationship between quantisation and test-generation quality.

5.7 Failure Modes and Denominator Effects

Failure analysis shows where generated tests stop being usable. In the main comparison, final generation failures range from 56 files for Qwen 27B repository-aware generation to 383 for the Qwen 9B AWQ generic baseline, again showing the executability advantage of repository-aware generation.

Table 5.7: Diagnostic generation failure modes for the main matched routes

Route	Pipeline	Failed n	Import	Interface	Assertion	Invalid	Runtime	Timeout	Other
Qwen 27B	Repository-aware	56	15	8	16	0	7	2	8
Qwen 27B	Generic baseline	95	0	11	37	0	21	2	24
Qwen 9B	Repository-aware	261	38	30	97	7	41	6	42
Qwen 9B	Generic baseline	330	8	35	114	2	60	10	101
Qwen 9B AWQ	Repository-aware	276	27	30	106	17	50	2	44
Qwen 9B AWQ	Generic baseline	383	11	37	145	5	61	12	112

Notes: Diagnostic categories were created based on the last recorded reason a test failed. These categories are used for grouping purposes only. The categorization of import, interface, assertion, invalid output and runtime errors follows the criteria in Appendix Table A.13. Source: `all_file_results.csv`; See Appendix Table A.14.

The data in Table 5.7 indicate that failures are not explained by a single cause. Assertion mismatches are common, which is consistent with the oracle limitation, while interface-related and import-related failures continue to occur even with repository-aware test construction. This suggests that the deterministic test harness and bounded symbol context reduce some opportunities for repository-context-related failures, but do not completely prevent them. It also supports the point that evaluation should not be limited to pass rate alone. The remaining failures include oracle-related assertion mismatches as well as import, interface and runtime problems.

Although coverage recomputation failed more frequently than mutation-stage processing, most accepted tests that reached mutation processing still led to viable mutation results. There were between 126 and 169 accepted tests that failed or timed out during subsequent coverage recomputation (Table 5.2; Appendix Table A.11). By contrast, the `Qwen/Qwen3.5-27B` repository-aware route produced 1,080 viable mutation results from 1,088 mutation candidates. This means that although the narrower denominator used for mutation analysis still matters, coverage recomputation accounts for the larger post-processing loss.

Further, as indicated by the failure analysis above, the oracle limitation reduces the ability of the pipeline to provide automatic semantic judgements. The pipeline has no way of determining whether a generated test fails because it is incorrect, or whether it has identified an existing problem with the source code. Therefore, in this controlled evaluation, `TheAlgorithms/Python` is treated as the reference behaviour, and failing tests are logged rather than included in either coverage or mutation analysis.

5.8 Answering the Research Questions

This section answers the six research questions.

RQ1 asked how effectively `LazyTest` can generate executable `pytest` tests for `TheAlgorithms/Python`. In summary, the three relevant high-performing routes were:

- the highest-performing repository-aware `Qwen` route, which produced passing generated test suites for 1,106 of 1,144 target files;
- the second route, using non-repeated `Qwen 27B`, which passed 1,088 target files;
- the third route, through `Gemini`, which passed 1,037 target files.

This indicates a good level of executable test generation under favourable repository conditions.

RQ2 asked whether bounded repair improves test pass rate. The results indicate that bounded repair improves test pass rates. The improvement is most visible where initial generation was weakest. Therefore, repair should be viewed as a tool to increase test pass rates, i.e., it acts as an executability mechanism and does not show that repaired tests are semantically superior.

RQ3 examined how generic and repository-aware generation differ. Repository-aware generation achieved higher initial and final pass rates in all matched comparisons, although generic generation sometimes achieved higher accepted-test coverage

or mutation scores. Thus, LazyTest is stronger as a full-target-set generation route; however, it does not uniformly produce stronger results on every accepted-test metric, such as test coverage.

RQ4 examined if generated tests show fault-revealing behaviour beyond structural coverage. The main LLM routes achieved viable mutation scores of roughly 67% to 72%. Furthermore, the use of mutation testing does not yield identical rankings of routes compared to coverage. Therefore, mutation testing provides additional information relative to coverage alone.

RQ5 examined what import and context limitations remain with respect to full-corpus generation. While the combined repository-aware condition improved executability, it still could not completely remove generation failures for certain targets or later failures during coverage recomputation for previously accepted tests.

RQ6 was secondary. As indicated above, the results indicate that different routes have different levels of performance; specifically: Gemini can be used as an API back-end within the same pipeline as the remaining routes; Pynguin has lower performance in this configuration; AWQ Qwen routes remain empirically viable although they do not provide a general model ranking. Runtime and attempt data in Table 5.3 are practical execution evidence rather than hardware-normalised speed measurements.

5.9 Chapter Summary

The evaluation shows that the combined repository-aware LazyTest condition improves executability across the main matched comparisons. Its strongest result is not uniformly higher accepted-test mutation score, but broader full-target-set executability and lower repair burden. Mutation-aware prompting improves viable mutation score in Qwen 27B runs, but usually reduces final pass rate. Gemini works as an API back-end inside the same pipeline, Pynguin is weaker under the evaluated configuration, and AWQ routes remain usable in several Qwen conditions without supporting broad quantisation claims.

Overall, the results support the controlled case-study claim. Under favourable TheAlgorithms/Python conditions, repository-aware LazyTest is associated with higher full-target-set executability and stronger target-normalised measurement than matched generic baselines. The results do not establish direct generalisation to arbitrary Python repositories; they identify the value and limits of LazyTest’s bounded repository-aware design.

Chapter 6

Conclusion and Future Work

6.1 Overview

This dissertation developed and evaluated LazyTest, a CI-integrated, execution-aware pipeline for automated Python unit-test generation. The empirical evaluation was framed as a controlled case study focused on TheAlgorithms/Python, rather than as a broad benchmark across Python repositories. This scope reflects the evidence available for the final evaluation, since reliable completed empirical results were not obtained for dependency-rich repositories such as Requests, Pandas or NumPy. The dissertation therefore addressed a more bounded question: how far a repository-aware LLM test-generation pipeline can improve generated-test executability and measurable test quality under favourable pure-Python repository conditions.

Two related artefacts were produced. The first was a pull-request-facing GitHub Actions workflow, which shows how automated test generation can be requested, validated, reported and optionally committed within a repository workflow (Chapter 4; Appendix B, Table B.1; Listing B.1). The second was a separate local empirical pipeline used to generate the controlled TheAlgorithms/Python evaluation data analysed in Chapter 5, with supporting provenance and implementation evidence in Table A.1 and Listings B.2–B.7. This separation was important: the workflow provides CI and pull-request integration evidence, while the local pipeline provides the empirical evidence used to answer the research questions.

6.2 Answers to the Research Questions

Research Question 1 asked how effectively LazyTest can generate executable pytest tests for TheAlgorithms/Python within a controlled case study. Results show high final pass rates under the favourable conditions of this repository. The highest-performing repository-aware Qwen route produced passing generated test suites for 1,106 of 1,144 selected target files, as seen in Table 5.4 and Appendix Table A.5. The primary non-repeated Qwen/Qwen3.5-27B repository-aware route passed validation for 1,088 of the 1,144 selected targets, as reported in Table 5.1, Table 5.2, and Appendix Table A.5. Additionally, the Gemini API backend passed validation for 1,037 of the 1,144 selected targets when run as a backend within the same repository-aware pipeline, as reported in Table 5.4 and Appendix Table A.5. These results indicate that LazyTest can generate executable tests for a significant portion of the selected target set of TheAlgorithms/Python. They do not, however, show that these levels of reliability would be replicated across other repositories with heavy dependency management, compiled components, complex fixtures or project-specific runtime configurations.

Research Question 2 examined whether bounded execution-aware repair would improve generated-test executability and validation success. Results showed that it did: final pass rate was higher than inferred initial pass rate after bounded repair, as seen in Table 5.1. The largest improvements were seen in the generic baseline routes, which began with lower inferred initial pass rates and left more failures for the repair loop to address. This supports the use of pytest feedback and tracebacks to enhance executability. However, repair remains an executability mechanism rather than proof of improved test quality: it may fix an import or signature issue, but it may also weaken an assertion or avoid a difficult execution path.

Research Question 3 compared generic target-file generation to repository-aware LazyTest generation. As seen in Table 5.1, all three matched main evaluations demonstrated that repository-aware LazyTest generation achieved higher initial and final pass rates than the generic baseline routes. This is the central empirical finding supporting the main repository-aware versus generic baseline comparison. However, repository-aware generation did not exceed the generic baseline on each measure among accepted tests. In some cases, the generic baseline exceeded repository-aware generation in terms of evaluated coverage and viable mutation score among the subset of generated tests that survived through subsequent evaluation. Thus, instead of concluding that repository-aware generation consistently generates individually stronger accepted

tests, a more accurate conclusion is that the repository-aware pipeline generated executable tests for more files in the overall target set and required less repair. For a repository-facing pipeline this represents an important benefit since non-executable generated tests have very little practical utility.

Research Question 4 examined whether generated tests showed fault-revealing ability beyond structural coverage. The mutation testing provided additional comparative evidence beyond smoke-test execution alone. Viable mutation scores were approximately 66–74 percent for the main LLM routes. Moreover, the mutation-aware Qwen/Qwen3.5-27B prompt variants produced higher viable mutation scores than the corresponding base-prompt routes, as shown in Table 5.1 and Table 5.5. Structural coverage and viable mutation score were related, although they did not rank every route identically. Thus, mutation testing provides a useful supplemental method of evaluating generated test quality beyond structural coverage. Nonetheless, viable mutation score represents an empirical indicator of fault-revealing ability within the mutation-evaluated subset and does not establish the intended semantic correctness of the target implementation.

Research Question 5 addressed the remaining remaining limitations in import handling and repository-context extraction. Although the repository-aware condition, which consists of AST-constrained prompting, deterministic import handling, import stripping, and restricted access to `target_module`, achieved greater executability than the corresponding generic target-file baselines as indicated by the matched comparisons, the design does not isolate individual mechanisms contributing to this increase in executability. Furthermore, some target files continued to fail during generation while others failed later during recomputation of structural coverage. Failures occurring during evaluation included assertion mismatch errors, interface failure errors, import handling errors, runtime exception errors and timeout errors, as indicated in Table 5.7. Thus, the conclusion remains bounded: the combined repository-aware condition used in this case study resulted in enhanced generated-test executability; however, it did not eliminate repository-level test-generation failures.

Research Question 6 was secondary and concerned route-level differences. The Gemini route shows that Gemini can be used as an API backend within the same repository-aware LazyTest pipeline, while several local open-weight routes produced strong route-level results under the evaluated configurations. Pynguin performed below the selected LLM-based LazyTest routes under the acceptance criteria and Pynguin configuration used here, particularly on viable mutation score (Table 5.4); this should

not be generalised to Pynguin overall. The AWQ results are also bounded: Table 5.6 shows that local AWQ-quantised routes remained usable in the assessed configurations, but does not imply a universal causal link between quantisation and generated-test quality.

6.3 Contributions and Achievements

The project provides four key contributions.

First, it developed a pull-request-facing automated test-generation workflow. The GitHub Actions layer shows how generated tests may be requested, validated, reported and optionally committed within a repository workflow, rather than created only through an offline script. The workflow demonstrated here supports the CI-integration objective of this dissertation while remaining separate from the local empirical pipeline from which the Chapter 5 data were collected.

Second, it developed a repository-aware LLM generation pipeline for Python unit tests. LazyTest includes several components, including target discovery; repository-aware prompt construction; AST-based symbol extraction; deterministic import handling; output sanitisation; and controlled access to the `target_module`, as detailed in Appendix B, Table B.1 and Listings B.2–B.3. As such, LazyTest is best viewed as an execution-aware pipeline and not simply as a single-prompt wrapper over an LLM.

Third, it developed a bounded validation and repair process for generated tests. Rather than treating each generated test only as a final success or failure, generated tests are run against the codebase using `pytest`. Failed tests provide feedback for later repair attempts within the bounded process. Failed outcomes are also recorded rather than treated as successful outputs. Additionally, the same local evaluation pipeline supports coverage recomputation, mutation testing and failure-mode analysis, as shown in Listings B.4–B.7 and Appendix Tables A.13 and A.14.

Fourth, it provided a controlled TheAlgorithms/Python case-study evaluation using several complementary metrics. Rather than evaluating generated tests solely through coverage or final pass rate, the evaluation included initial pass rate; final pass rate; repair improvement; evaluated coverage; target-normalised coverage; viable mutation score; runtime evidence; and diagnostic information about why tests failed. These additional metrics allowed for the identification of trade-offs. For example, repository-aware generation produced higher full-target-set executability than the generic baseline routes. In contrast, mutation-aware prompts produced higher viable mutation scores in

Qwen/Qwen3.5-27B prompt-variant runs, but at some cost to final pass rate.

6.4 Critical Reflection and Limitations

The most important limitation is the case-study scope. As mentioned above, TheAlgorithms/Python was selected because it offered favourable conditions for an initial evaluation. Many modules within TheAlgorithms/Python are algorithmic, written in Python, and often have clearer input-output behaviour than dependency-heavy repositories. These conditions allowed the project to evaluate import handling, bounded context construction, and repair based upon execution without having to address all of the issues related to complex dependency installation, compiled extensions, large fixture sets, and project-specific testing conventions. However, the results from this evaluation should not be generalised directly to arbitrary Python repositories.

A second limitation concerns causality. The primary comparison is at the pipeline level rather than a set of ablated comparisons (e.g., how much each part of the pipeline contributes to success). The repository-aware condition examined in this research includes multiple mechanisms including AST-derived symbol information, import restrictions, deterministic preamble construction, sanitised output, controlled access to targets, and bounded repair. While Chapter 5 shows that the combination of these conditions is associated with higher full-target-set executability than the generic baseline, it does not demonstrate the isolated contributions of each mechanism. To understand which mechanisms contribute the most to pass rates, coverage, and mutation scores, a more detailed ablation would need to be performed.

Thirdly, a further limitation concerns the test oracle problem. In evaluating generated tests for TheAlgorithms/Python, the current repository implementation was used as the behavioural reference. This approach provides a way to perform automated validation and a basis for comparing expected versus actual behaviour. However, it does not prove the semantic correctness of generated tests. Instead, passing generated tests should be understood as preserving existing observable behaviour and potentially revealing regressions. Additionally, while mutation testing strengthens the evaluation by checking whether tests detect artificially modified behaviour, it does not eliminate the test oracle problem.

Fourthly, while runtime metrics were recorded for both local and API-based backends, they were not normalised across different hardware configurations. These measurements should therefore be viewed as indicative of the practical costs associated

with executing the evaluated pipelines. However, they are not sufficient for establishing definitive conclusions about model speed, service efficiency, or architectural quality.

Finally, the failure-mode categories are broad rather than definitive. These categories are meant to assist in identifying overall patterns of failures, including import failures, interface errors, assertion mismatches and timeouts. However, some types of failures could logically fall into more than one category. The classification therefore serves as an interpretive tool for understanding limitations but is not suitable for providing a comprehensive root-cause analysis for all failed targets.

6.5 Future Work

The most important future direction is to test whether LazyTest can be extended beyond favourable pure-Python repositories. This should not mean applying the same scripts unchanged. Dependency-heavy repositories would require additional environment management, dependency identification, fixture handling, package-layout analysis and repository-specific test-convention detection. A suitable next stage would therefore select several contrasting repository types, such as a web application or framework, a data-science library and a command-line application, each with its own justified evaluation methodology. This would make it possible to study whether the pipeline degrades gradually with repository complexity, or whether particular dependency and fixture structures cause different failure modes.

A second direction is finer-grained ablation. Future work should isolate AST-based symbol extraction, deterministic import preamble construction, import stripping, prompt constraints, prompt repetition and execution-aware repair. This would help identify which mechanisms mainly affect first-pass executability, repair success, coverage and viable mutation score.

A third direction is stronger oracle support. For algorithmic code, future versions of LazyTest could incorporate metamorphic testing, property-based testing or reference implementations where available. These methods could reduce reliance on the current implementation as the behavioural reference and help separate useful characterisation tests from tests that preserve incorrect behaviour.

A fourth direction is improved repository-context retrieval and environment management. The current system uses bounded target-file context, AST-derived symbols

and a controlled execution interface. Future versions could add language-server information, import graphs, approximate call graphs, sibling-module retrieval, or retrieval over existing human-written tests. However, this should remain bounded, since excessive or irrelevant context may distract the model. For larger repositories, LazyTest would also need to infer optional dependencies, compiled components, generated files, configuration files and project-specific setup steps before generated tests could be evaluated reliably.

Finally, the pull-request-facing workflow should be evaluated with human users. A maintainer-facing study could examine whether generated tests are understandable, whether pull-request comments are useful, how much manual editing is needed, and whether maintainers would accept any generated tests into a real project. This would move the evaluation beyond executable test generation and towards practical usefulness in open-source review.

6.6 Final Conclusion

LazyTest provides a controlled case-study artefact for investigating repository-aware LLM-based unit-test generation in Python. Under favourable TheAlgorithms/Python conditions, the repository-aware LazyTest pipeline was associated with higher full-target-set executability than the matched generic target-file baselines, and bounded repair increased validation success across the main evaluated conditions. The strongest evidence is not that generated tests prove correctness, nor that the approach generalises directly to all Python repositories. Rather, the dissertation shows that, within this controlled pure-Python case study, the combined repository-aware LazyTest pipeline was associated with more executable and more measurable LLM-generated pytest tests than the matched generic target-file baselines.

The final conclusion is therefore deliberately bounded. LazyTest demonstrates the practical potential, within this case study, of combining LLM generation with repository-aware import handling, execution feedback and post-generation evaluation. It also demonstrates why these mechanisms remain incomplete. Even under favourable conditions, generated tests may fail, repairs may be weak, accepted tests may later fail during coverage recomputation, and mutation scores apply only to the mutation-evaluated subset. The project therefore contributes both a working pipeline and an evidence-based account of its limits, providing a foundation for future work on more complex Python repositories.

Bibliography

- [1] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: generating tests from issues to validate swe patches. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML’25. JMLR.org, 2025.
- [2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, FSE 2024, page 185–196, New York, NY, USA, 2024. Association for Computing Machinery.
- [3] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Trans. Softw. Eng.*, 41(5):507–525, May 2015.
- [4] Moritz Beller, Georgios Gousios, Annibale Panichella, and Andy Zaidman. When, how, and why developers (do not) test in their ides. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, page 179–190, New York, NY, USA, 2015. Association for Computing Machinery.
- [5] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, FSE 2024, page 572–576, New York, NY, USA, 2024. Association for Computing Machinery.
- [6] Ermira Daka, José Campos, Gordon Fraser, Jonathan Dorn, and Westley Weimer. Modeling readability to improve unit tests. In *Proceedings of the 2015 10th*

- Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, page 107–118, New York, NY, USA, 2015. Association for Computing Machinery.
- [7] Ermira Daka and Gordon Fraser. A survey on unit testing practices and problems. In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, ISSRE '14, page 201–211, USA, 2014. IEEE Computer Society.
- [8] Giovanni Grano, Fabio Palomba, Dario Di Nucci, Andrea De Lucia, and Harald C. Gall. Scented since the beginning: On the diffuseness of test smells in automatically generated test code. *J. Syst. Softw.*, 156(C):312–327, October 2019.
- [9] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. Usage, costs, and benefits of continuous integration in open-source projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ASE '16, page 426–437, New York, NY, USA, 2016. Association for Computing Machinery.
- [10] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE 2014, page 435–445, New York, NY, USA, 2014. Association for Computing Machinery.
- [11] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Trans. Softw. Eng.*, 37(5):649–678, September 2011.
- [12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Prompt repetition improves non-reasoning llms. *ArXiv*, abs/2512.14982, 2025.
- [13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 02 2024.
- [14] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. Graphcoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 570–581, New York, NY, USA, 2024. Association for Computing Machinery.

- [15] Stephan Lukasczyk and Gordon Fraser. Pynguin: automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ICSE '22*, page 168–172, New York, NY, USA, 2022. Association for Computing Machinery.
- [16] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. Automated unit test generation for python. In *Search-Based Software Engineering: 12th International Symposium, SSBSE 2020, Bari, Italy, October 7–8, 2020, Proceedings*, page 9–24, Berlin, Heidelberg, 2020. Springer-Verlag.
- [17] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. An empirical study of automated unit test generation for python. *Empirical Softw. Engg.*, 28(2), January 2023.
- [18] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering*, 44:122–158, 02 2018.
- [19] Goran Petrović and Marko Ivanković. State of mutation testing at google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP '18*, page 163–171, New York, NY, USA, 2018. Association for Computing Machinery.
- [20] Goran Petrović, Marko Ivanković, Gordon Fraser, and René Just. Practical mutation testing at scale: A view from google. *IEEE Trans. Softw. Eng.*, 48(10):3900–3912, October 2022.
- [21] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Softw. Engg.*, 14(2):131–164, April 2009.
- [22] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Trans. Softw. Eng.*, 50(1):85–105, January 2024.
- [23] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C. Gall, and Alberto Bacchelli. On the effectiveness of manual and automatic unit

- test generation: ten years later. In *Proceedings of the 16th International Conference on Mining Software Repositories*, MSR '19, page 121–125. IEEE Press, 2019.
- [24] TheAlgorithms/Python Contributors. Contributing Guidelines. <https://github.com/TheAlgorithms/Python/blob/791deb40f93c65a020f2c75e41b55c97ad566cc0/CONTRIBUTING.md>, n.d. Accessed: 30 April 2026.
- [25] TheAlgorithms/Python Contributors. TheAlgorithms/Python Repository. <https://github.com/TheAlgorithms/Python>, n.d. Accessed: 30 April 2026.
- [26] Mairieli Wessel, Ahmad Abdellatif, Igor Wiese, Tayana Conte, Emad Shihab, Marco A. Gerosa, and Igor Steinmacher. Bots for pull requests: the good, the bad, and the promising. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, page 274–286, New York, NY, USA, 2022. Association for Computing Machinery.
- [27] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, page 819–831, New York, NY, USA, 2024. Association for Computing Machinery.
- [28] Xin Yin, Chao Ni, Xinrui Li, Liushan Chen, Guojun Ma, and Xiaohu Yang. Enhancing llm’s ability to generate more repository-aware unit tests through precise context injection. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, page 623–635. IEEE Press, 2025.
- [29] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. *Proc. ACM Softw. Eng.*, 1(FSE), July 2024.
- [30] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, Singapore, December 2023. Association for Computational Linguistics.

Appendix A

Evaluation Artefacts and Supplementary Results

This appendix provides supplementary evidence for the Chapter 5 evaluation. It is included to keep the main results chapter compact while preserving the route mappings, metric definitions, denominator checks and source-data provenance used in the analysis. The raw CSV files are not reproduced in full because several of them are too wide for a readable printed appendix. Instead, the appendix reports compact audit tables derived from those files.

A.1 Evaluation Source Files and Aggregation Provenance

Table A.1 summarises the main evaluation artefacts used to construct Chapter 5.

Table A.1: Chapter 5 source-data provenance

Artefact	File(s)	Role in Chapter 5
Schema and join report	schema_report.txt	Records the loaded 1,144-file target set, the detected generation, coverage and mutation CSV schemas, and join-validation checks used to audit the aggregation.
Run-level summary	run_summary.csv	Run-level pass-rate, repair, coverage, mutation, runtime and timeout-retry summary fields. Used as a compact audit source for route-level values.
Condition-level summary	condition_summary.csv	Main condition-level aggregate table used for the Chapter 5 route comparisons, denominator tables, prompt-variant checks and runtime summaries.
Main comparison summary	main_pipeline_comparison.csv	Source table for the matched generic-baseline versus repository-aware LazyTest comparison reported in Chapter 5.
Secondary route summary	secondary_backend_summary.csv	Source table for secondary backend, prompt-variant and route-level comparisons that are not part of the central matched baseline comparison.
Joined per-file results	all_file_results.csv	Per-target joined generation, coverage and mutation evidence. Used for denominator checks and diagnostic failure-mode grouping.
Failure summaries	failure_summary.csv; generation_failure_summary.csv; coverage-failure-summary.csv; mutation-failure-summary.csv	Aggregated failure summaries for generation, coverage and mutation stages. These support the failure-mode and denominator discussions in Chapter 5.
Table workbook	chapter5_tables.xlsx	Spreadsheet copy of the main Chapter 5 and appendix tables used as a formatting and audit aid.
Aggregation warnings	warnings.log	Records aggregation caveats, including inferred initial pass-rate construction and missing retry-mutation artefacts for some routes.
Chapter figures	fig_main_pipeline_comparison.pdf; fig_qwen27b_prompt_variants.pdf; fig_selected_secondary_routes.pdf; fig_awq_matched_pass_rate.pdf	Figure artefacts generated from the Chapter 5 summary tables.

A.2 Metric Definitions

Table A.2 defines the main metrics used in Chapter 5. The definitions are included here because the evaluation uses different denominators for generation, coverage and mutation analysis.

Table A.2: Chapter 5 metric definitions

Metric	Definition used in Chapter 5	Interpretation and caveat
Generation target set	The fixed set of 1,144 discovered TheAlgorithms/Python source files after the technical filters defined in Chapter 3.	Used as the denominator for pass-rate analysis. It is a within-repository case-study target set, not a sample from all Python repositories.
Initial pass rate	Percentage of target files with final status <code>ok</code> and <code>attempts <= 1</code> .	This is a first-attempt proxy inferred from the available attempt-count field, not a separately stored first-attempt status column.
Final pass rate	Percentage of the 1,144 target files for which the final generated test suite passed validation.	Main executability metric. Failed or missing generations remain in the denominator.
Repair improvement	Final pass rate minus initial pass rate, reported in percentage points.	Measures improvement in executability after bounded repair. It does not by itself show that repaired tests are semantically stronger.
Coverage-evaluated set	Accepted generated tests that could be re-run by the coverage stage.	Narrower than the generation target set because tests must remain executable during later post-processing.
Evaluated combined coverage	Mean combined coverage over accepted generated tests that reached coverage evaluation.	Useful for accepted-test quality, but sensitive to survivor bias because failed generations are not included.
Target-normalised combined coverage	Combined coverage with missing or failed coverage results treated as zero over the full 1,144-file target set.	Used to reduce denominator bias when comparing routes with different pass rates.
Mutation-candidate set	Generated tests that were discoverable and mappable to a source file for primary mutation analysis.	Narrower than the target set because mutation testing requires an executable generated test and a valid source-test mapping.
Primary mutation-viable set	Mutation candidates whose mutation sessions produced viable killed or survived mutants.	Main denominator for viable mutation score. Timeout, incompetent and tool-error cases are not treated as viable mutants.
Viable mutation score	$K/(K+S) \times 100$, where K is killed viable mutants and S is survived viable mutants.	Main mutation metric used in Chapter 5. It is an empirical fault-revealing indicator, not proof of semantic correctness.
Summed recorded generation time	Sum of per-file generation, validation and repair times recorded by the local scripts.	Practical cost evidence. It is not a hardware-normalised wall-clock or model-speed benchmark because routes used local serving and parallel execution.

A.3 Route, Model and Prompt Label Mapping

Table A.3 defines the shortened labels used in Chapter 5. Internal run names are retained in the CSV artefacts, while the dissertation uses normalised labels for readability.

Table A.3: Chapter 5 route, model and prompt-label mapping

Dissertation label	Internal or exact identifier		Use in Chapter 5
Qwen 27B	Qwen/Qwen3.5-27B;	condition prefix qwen_3.5	Main and secondary local LLM route.
Qwen 9B	Qwen/Qwen3.5-9B;	condition prefix qwen_3.5_9b	Main and secondary local LLM route.
Qwen 27B AWQ	Condition prefix qwen_3.5_cyankiwi		Local AWQ route used for route-level and prompt-variant robustness checks.
Qwen 9B AWQ	Condition prefix qwen_3.5_9b_cyankiwi		Main AWQ route and matched generic-baseline comparison.
Qwen3-Coder-Next AWQ	Condition prefix qwen_coder_next		Selected secondary AWQ route.
Devstral AWQ	Condition prefix devstral_cyankiwi		Selected secondary AWQ route from a different model family.
Gemini 3.1 Flash Lite	gemini-3.1-flash-lite-preview		API backend inside the repository-aware LazyTest pipeline.
Pynguin	pynguin/run_2		Search-based generation route under the configuration used in this evaluation.
Repository-aware LazyTest	repository_aware_lazytest		Local pipeline using AST-derived symbols, deterministic import handling and bounded pytest repair.
Generic target-file baseline	generic.target_file_baseline		Simpler LLM baseline without the repository-aware import harness and target-symbol context.
Repository-aware prompt	base	base_prompt; base_prompt.long_traceback	Prompt without explicit mutation-score or high-coverage metric target wording.
Repository-aware prompt with shortened traceback	base	base_prompt.short_traceback	Base prompt variant using a shorter traceback form.
Strict mutation-aware prompt		mut_not_fixed; mutation_aware.notfixed.long_trace	Prompt variant using explicit high-coverage and mutation-score target wording, including the earlier 100% line-coverage phrasing.
Softened mutation-aware prompt		mut_fixed; mutation_aware.fixed.long_trace	Prompt variant asking the model to maximise branch and line coverage and prioritise mutation score without the absolute 100% wording.
Prompt repetition enabled	repeat; prompt_repetition=yes		Run-level prompt variant in which the prompt was repeated.
Prompt repetition disabled	no_repeat; prompt_repetition=no		Run-level prompt variant without prompt repetition.

A.4 Main Matched Route Source Values

Table A.4 reproduces the compact source values for the main matched comparison. It is included here to provide an appendix audit trail for Table 5.1.

Table A.4: Main matched route source values

Route	Pipeline	Initial pass	Final pass	Repair	Target-norm. cov.	Viable mut.	Mut. n
		%	%	pp	%	%	
Qwen 27B	Repository-aware	72.90	95.10	22.20	64.86	70.47	1080
Qwen 27B	Generic baseline	33.74	91.70	57.95	63.01	73.86	1040
Qwen 9B	Repository-aware	48.43	77.19	28.76	49.86	68.63	878
Qwen 9B	Generic baseline	29.81	71.15	41.35	47.12	72.88	810
Qwen 9B AWQ	Repository-aware	44.93	75.87	30.94	48.90	65.86	864
Qwen 9B AWQ	Generic baseline	31.03	66.52	35.49	44.00	70.92	758

Notes: Values are derived from `main_pipeline_comparison.csv` and `condition_summary.csv`.

The main comparison is pipeline-level rather than an ablation of individual mechanisms.

A.5 All Route Summary

Table A.5 gives a compact summary of all route conditions used in the Chapter 5 aggregation. It is intended as an audit table, not as a single model leaderboard.

Table A.5: All route compact metric summary

Route / run	Model/tool	Pipeline	Prompt / condition	Ok	Pass %	T-cov. %	E-cov. %	Mut. %
qwen_3.5/run_1	Qwen 27B	Repo-aware	Softened mut.-aware + repetition	1030	90.03	62.65	82.01	74.10
qwen_3.5/run_2	Qwen 27B	Repo-aware	Strict mut.-aware + repetition	1012	88.46	61.39	82.05	74.08
qwen_3.5/run_3	Qwen 27B	Repo-aware	Base + repetition	1106	96.68	66.29	80.93	70.60
qwen_3.5/run_4	Qwen 27B	Repo-aware	Base short traceback + repetition	1101	96.24	66.03	80.88	70.80
qwen_3.5/run_5	Qwen 27B	Repo-aware	Strict mut.-aware + repetition	1055	92.22	64.16	82.00	74.01
qwen_3.5/run_6	Qwen 27B	Repo-aware	Softened mut.-aware + repetition	1067	93.27	64.48	81.96	74.23
qwen_3.5/run_7	Qwen 27B	Repo-aware	Base	1088	95.10	64.86	80.74	70.47
qwen_3.5/run_8	Qwen 27B	Repo-aware	Strict mut.-aware	995	86.98	60.88	81.74	73.92
qwen_3.5/run_9	Qwen 27B	Repo-aware	Softened mut.-aware	1024	89.51	62.13	81.51	73.30
qwen_3.5.baseline/run_10	Qwen 27B	Generic	Generic baseline	1049	91.70	63.01	81.91	73.86
qwen_3.5.cyankiwi/run_1	Qwen 27B AWQ	Repo-aware	Base	1076	94.06	64.26	80.61	70.10
qwen_3.5.cyankiwi/run_2	Qwen 27B AWQ	Repo-aware	Base + repetition	1101	96.24	65.24	80.43	70.30
qwen_3.5.cyankiwi/run_3	Qwen 27B AWQ	Repo-aware	Strict mut.-aware	970	84.79	59.74	81.95	73.39
qwen_3.5.cyankiwi/run_4	Qwen 27B AWQ	Repo-aware	Strict mut.-aware + repetition	1034	90.38	63.07	82.00	73.02
qwen_3.5.cyankiwi/run_5	Qwen 27B AWQ	Repo-aware	Softened mut.-aware	991	86.63	60.22	81.52	73.71
qwen_3.5.9b/run_1	Qwen 9B	Repo-aware	Base	883	77.19	49.86	76.87	68.63
qwen_3.5.9b/run_2	Qwen 9B	Repo-aware	Base + repetition	919	80.33	53.99	79.28	70.27
qwen_3.5.9b/run_3	Qwen 9B	Repo-aware	Strict mut.-aware	545	47.64	31.68	77.95	71.50
qwen_3.5.9b/run_4	Qwen 9B	Repo-aware	Strict mut.-aware + repetition	636	55.59	37.38	78.90	72.56
qwen_3.5.9b/run_5	Qwen 9B	Repo-aware	Softened mut.-aware	562	49.13	32.94	77.22	70.06
qwen_3.5.9b/run_6	Qwen 9B	Repo-aware	Softened mut.-aware + repetition	654	57.17	38.99	79.50	72.16
qwen_3.5.9b.baseline/run_10	Qwen 9B	Generic	Generic baseline	814	71.15	47.12	79.75	72.88
qwen_3.5.9b.cyankiwi/run_1	Qwen 9B AWQ	Repo-aware	Base	868	75.87	48.90	76.53	65.86
qwen_3.5.9b.cyankiwi/run_2	Qwen 9B AWQ	Repo-aware	Base + repetition	931	81.38	54.90	79.40	70.04
qwen_3.5.9b.cyankiwi/run_3	Qwen 9B AWQ	Repo-aware	Strict mut.-aware	454	39.69	26.11	76.97	69.72
qwen_3.5.9b.cyankiwi/run_4	Qwen 9B AWQ	Repo-aware	Strict mut.-aware + repetition	546	47.73	32.54	79.20	72.89
qwen_3.5.9b.cyankiwi.baseline/run_10	Qwen 9B AWQ	Generic	Generic baseline	761	66.52	44.00	79.28	70.92

Route / run	Model/tool	Pipeline	Prompt / condition	Ok	Pass %	T-cov. %	E-cov. %	Mut. %
qwen_coder_next/run_1	Qwen3-Coder-Next AWQ	Repo-aware	Base + repetition	1028	89.86	60.24	79.76	70.09
qwen_coder_next/run_3	Qwen3-Coder-Next AWQ	Repo-aware	Base + repetition	1027	89.77	59.94	79.36	69.92
qwen_coder_next/run_4	Qwen3-Coder-Next AWQ	Repo-aware	Strict mut.-aware + repetition	747	65.30	42.26	76.61	68.62
qwen_coder_next/run_5	Qwen3-Coder-Next AWQ	Repo-aware	Softened mut.-aware + repetition	787	68.79	44.51	75.66	68.06
devstral_cyankiwi/run_1	Devstral AWQ	Repo-aware	Base + repetition	897	78.41	53.79	80.75	71.35
devstral_cyankiwi/run_2	Devstral AWQ	Repo-aware	Base	882	77.10	52.11	80.12	71.33
devstral_cyankiwi/run_3	Devstral AWQ	Repo-aware	Strict mut.-aware	420	36.71	26.24	79.62	70.55
devstral_cyankiwi/run_5	Devstral AWQ	Repo-aware	Softened mut.-aware	507	44.32	31.36	80.63	71.20
gemini-3.1-flash-lite- preview/run_10	Gemini 3.1 Flash Lite	Repo-aware	Combined API batches	1037	90.65	62.39	81.11	72.22
pynguin/run_2	Pynguin	Pynguin	Search-based	693	60.58	38.01	71.52	40.65

Notes: T-cov. is target-normalised combined coverage over the full 1,144-file target set. E-cov. is evaluated combined coverage over accepted generated tests that reached coverage evaluation. Mut. is viable mutation score. The shortened route/run labels are display labels only; full internal run names are retained in the CSV artefacts.

A.6 All Route Denominator Summary

Table A.6 gives a compact denominator table for all evaluated route conditions. It is included to make clear which routes retain targets through generation, coverage and mutation post-processing.

Table A.6: All route denominator summary

Route / run	Target	Final	Cov.	Cov.	Mut.	Mut.
	ok	eval.	failed	cand.	viable	
qwen.27b/run.1	1144	1030	1030	156	1030	1022
qwen.27b/run.2	1144	1012	1012	156	1012	1005
qwen.27b/run.3	1144	1106	1106	169	1106	1099
qwen.27b/run.4	1144	1101	1101	167	1101	1093
qwen.27b/run.5	1144	1055	1055	160	1055	1048
qwen.27b/run.6	1144	1067	1067	167	1067	1057
qwen.27b/run.7	1144	1088	1088	169	1088	1080
qwen.27b/run.8	1144	995	995	143	995	989
qwen.27b/run.9	1144	1024	1024	152	1024	1011
qwen.27b_generic/run.10	1144	1049	1049	169	1049	1040
qwen.27b_awq/run.1	1144	1076	1076	164	1076	1069
qwen.27b_awq/run.2	1144	1101	1101	173	1101	1093
qwen.27b_awq/run.3	1144	970	970	136	970	963
qwen.27b_awq/run.4	1144	1034	1034	154	1034	1024
qwen.27b_awq/run.5	1144	991	991	146	991	983
qwen.9b/run.1	1144	883	883	141	883	878
qwen.9b/run.2	1144	919	919	140	919	911
qwen.9b/run.3	1144	545	545	80	545	540
qwen.9b/run.4	1144	636	636	94	636	629
qwen.9b/run.5	1144	562	562	74	562	555
qwen.9b/run.6	1144	654	654	93	654	649
qwen.9b_generic/run.10	1144	814	814	138	814	810
qwen.9b_awq/run.1	1144	868	868	137	868	864
qwen.9b_awq/run.2	1144	931	931	140	931	923
qwen.9b_awq/run.3	1144	454	454	66	454	448
qwen.9b_awq/run.4	1144	546	546	76	546	539
qwen.9b_awq_generic/run.10	1144	761	761	126	761	758
coder_next_awq/run.1	1144	1028	1028	164	1028	1021
coder_next_awq/run.3	1144	1027	1027	163	1027	1021
coder_next_awq/run.4	1144	747	747	116	747	743
coder_next_awq/run.5	1144	787	787	114	787	783
devstral_awq/run.1	1144	897	897	135	897	892
devstral_awq/run.2	1144	882	882	138	882	878
devstral_awq/run.3	1144	420	420	43	420	418
devstral_awq/run.5	1144	507	507	62	507	500
gemini/run.10	1144	1037	1037	157	1037	1028
pynguin/run.2	1144	693	693	85	693	660

Notes: Final ok is the number of targets for which the final generated test suite passed validation. Coverage-evaluated and coverage-failed counts refer to the separate coverage recalculation stage. Mutation candidates and viable mutation cases refer to the primary mutation analysis. Timeout-retry artefacts are treated as diagnostic evidence and are not merged into the primary viable mutation-score denominator. Route/run labels are shortened display identifiers; full internal run names are retained in the source CSV artefacts and mapped to dissertation labels in Table A.3.

A.7 Statement and Branch Coverage

Table A.7 reports separate statement, branch and combined coverage values for the main matched routes. Chapter 5 uses combined coverage in the main text for compactness, while this appendix table preserves the separate coverage components.

Table A.7: Statement and branch coverage for the main matched routes

Route	Pipeline	Eval. stmt. %	Eval. branch %	Eval. comb. %	Target stmt. %	Target branch %	Target comb. %
Qwen 27B	Repository-aware	68.17	79.99	80.74	64.83	64.26	64.86
Qwen 27B	Generic baseline	68.64	81.01	81.91	62.94	62.32	63.01
Qwen 9B	Repository-aware	64.69	75.83	76.87	49.93	49.19	49.86
Qwen 9B	Generic baseline	66.00	79.46	79.75	46.96	46.95	47.12
Qwen 9B AWQ	Repository-aware	64.60	75.22	76.53	49.02	48.06	48.90
Qwen 9B AWQ	Generic baseline	65.90	79.40	79.28	43.84	44.00	44.00

Notes: “Eval.” values are means over coverage-evaluated accepted tests. “Target” values treat missing or failed coverage results as zero over the full 1,144-file generation target set. Values are derived from `condition_summary.csv`.

A.8 Prompt-Variant Robustness Across Qwen Routes

Table A.8 extends the Chapter 5 Qwen 27B prompt-variant comparison with additional Qwen 9B and Qwen 27B AWQ route evidence. It is included as a robustness check rather than as a separate prompt-engineering benchmark.

Table A.8: Prompt-variant robustness across Qwen routes

Route	Prompt	Repeated	Final pass	Target cov.	Eval. cov.	Viable mut.	Mut. <i>n</i>
			%	%	%	%	
Qwen 27B	Base	No	95.10	64.86	80.74	70.47	1080
Qwen 27B	Base	Yes	96.68	66.29	80.93	70.60	1099
Qwen 27B	Strict mut.-aware	No	86.98	60.88	81.74	73.92	989
Qwen 27B	Strict mut.-aware	Yes	92.22	64.16	82.00	74.01	1048
Qwen 27B	Softened mut.-aware	No	89.51	62.13	81.51	73.30	1011
Qwen 27B	Softened mut.-aware	Yes	93.27	64.48	81.96	74.23	1057
Qwen 9B	Base	No	77.19	49.86	76.87	68.63	878
Qwen 9B	Base	Yes	80.33	53.99	79.28	70.27	911
Qwen 9B	Strict mut.-aware	No	47.64	31.68	77.95	71.50	540
Qwen 9B	Strict mut.-aware	Yes	55.59	37.38	78.90	72.56	629
Qwen 9B	Softened mut.-aware	No	49.13	32.94	77.22	70.06	555
Qwen 9B	Softened mut.-aware	Yes	57.17	38.99	79.50	72.16	649
Qwen 27B AWQ	Base	No	94.06	64.26	80.61	70.10	1069
Qwen 27B AWQ	Base	Yes	96.24	65.24	80.43	70.30	1093
Qwen 27B AWQ	Strict mut.-aware	No	84.79	59.74	81.95	73.39	963
Qwen 27B AWQ	Strict mut.-aware	Yes	90.38	63.07	82.00	73.02	1024
Qwen 27B AWQ	Softened mut.-aware	No	86.63	60.22	81.52	73.71	983

Notes: The Qwen 27B and Qwen 9B routes have complete base, strict mutation-aware and softened mutation-aware comparisons with and without prompt repetition. The Qwen 27B AWQ route is partial because the softened mutation-aware repeated condition was not available in the current summary. Values are derived from `condition_summary.csv` and `secondary_backend_summary.csv`.

A.9 Matched Full and AWQ Qwen Route Summary

Table A.9 gives the source values behind the matched full versus AWQ Qwen comparison in Chapter 5. These averages are route-level summaries over matched condition pairs and should not be interpreted as a controlled quantisation study.

Table A.9: Matched full and AWQ Qwen route summary

Matched route pair	Runs	Full pass	AWQ pass	Full eval. cov.	AWQ eval. cov.	Full mut. / AWQ mut.
		%	%	%	%	%
Qwen 27B vs Qwen 27B AWQ	5	92.10	90.42	81.39	81.30	72.46 / 72.10
Qwen 9B vs Qwen 9B AWQ	5	66.38	62.24	78.55	78.28	71.17 / 69.89

Notes: Values are averages over matched full/AWQ condition pairs. They are included as secondary route-level evidence only. Values are derived from `condition_summary.csv`.

A.10 Runtime and Repair-Attempt Evidence

Table A.10 reproduces the main runtime and repair-attempt evidence used in Chapter 5. These values are recorded generation-time measurements from the local scripts, not hardware-normalised model-speed benchmarks.

Table A.10: Runtime and repair-attempt evidence for the main matched routes

Route	Pipeline	Recorded gen. time h	Median per-file s	Mean attempts	Mean attempts ok	Tests generated
Qwen 27B	Repository-aware	22.46	41.31	1.51	1.34	9356
Qwen 27B	Generic baseline	96.36	181.84	2.24	2.00	18663
Qwen 9B	Repository-aware	122.62	183.36	2.43	1.67	9169
Qwen 9B	Generic baseline	123.06	246.11	2.90	2.05	15357
Qwen 9B AWQ	Repository-aware	145.45	222.08	2.55	1.77	8560
Qwen 9B AWQ	Generic baseline	196.98	452.51	2.99	1.98	13814

Notes: Recorded generation time is the summed per-file generation time reported by the local metrics. It should not be interpreted as wall-clock runtime because the runs used local serving and parallel processing. Values are derived from `condition_summary.csv` and `run_summary.csv`.

A.11 Coverage Rerun Failure Summary

Table A.11 reports coverage-stage failures for the main matched routes. This table supports the denominator caveat that a generated test may pass during the generation loop but fail later during coverage recalculation.

Table A.11: Coverage rerun failure summary for the main matched routes

Route	Pipeline	Final ok n	Coverage failed n	Failure rate %
Qwen 27B	Repository-aware	1088	169	15.53
Qwen 27B	Generic baseline	1049	169	16.11
Qwen 9B	Repository-aware	883	141	15.97
Qwen 9B	Generic baseline	814	138	16.95
Qwen 9B AWQ	Repository-aware	868	137	15.78
Qwen 9B AWQ	Generic baseline	761	126	16.56

Notes: Failure rate is coverage-failed n divided by final-ok n . Values are derived from `condition_summary.csv` and `coverage_failure_summary.csv`.

A.12 Mutation Denominator and Primary Mutation Outcomes

Table A.12 reports the mutation-analysis denominator values for the main matched routes. The main Chapter 5 mutation score is computed only over primary mutation-viable cases.

Table A.12: Mutation denominator and primary mutation outcome summary for the main matched routes

Route	Pipeline	Candidate	Row present	Completed	Viable	Baseline fail	Tool / missing	Viable mut. %
Qwen 27B	Repository-aware	1088	1087	1080	1080	6	2	70.47
Qwen 27B	Generic baseline	1049	1049	1040	1040	8	1	73.86
Qwen 9B	Repository-aware	883	883	878	878	5	0	68.63
Qwen 9B	Generic baseline	814	814	810	810	4	0	72.88
Qwen 9B AWQ	Repository-aware	868	866	864	864	2	2	65.86
Qwen 9B AWQ	Generic baseline	761	759	758	758	1	2	70.92

Notes: “Tool / missing” combines mutation-tool-error and incomplete-or-missing mutation rows where applicable. Timeout-retry artefacts are not merged into the primary viable mutation-score denominator. Values are derived from `condition_summary.csv` and `mutation_failure_summary.csv`.

A.13 Failure-Mode Classification Rules

Table A.13 defines the diagnostic grouping rules used for generation failure-mode analysis in Chapter 5. The categories are intended to support interpretation of recurring failure patterns; they are not definitive root-cause labels.

Table A.13: Generation failure-mode classification rules

Category	Indicative signals in the recorded failure note	Interpretation
Import	<code>ImportError</code> , <code>ModuleNotFoundError</code> , missing helper or missing dependency messages	The generated test or target execution could not resolve required modules, helpers or package context.
Interface	<code>AttributeError</code> , <code>TypeError</code> , wrong number of arguments, missing attribute, wrong target access pattern	The test reached the target interface but called it through an incorrect symbol, signature or object structure.
Assertion	<code>AssertionError</code> , expected-output mismatch, approximate-value mismatch	The generated test executed but the asserted behaviour did not match the repository behaviour under the evaluation environment.
Invalid output	Syntax errors, malformed Python, no valid <code>def test_...</code> function, invalid or empty generated test file	The model output could not be treated as a valid pytest test suite.
Runtime	Runtime exceptions such as <code>ValueError</code> , <code>IndexError</code> , <code>KeyError</code> , recursion errors or unexpected target exceptions	The generated test triggered an exception during execution that was not classified as an import/interface/assertion failure.
Timeout	Subprocess timeout, pytest timeout or long-running generated test execution	The generated or repaired test exceeded the configured execution budget.
Other	Failure notes that do not match the above rules, or are too ambiguous to classify confidently	Residual diagnostic category. It prevents over-interpreting noisy or incomplete failure messages.

Notes: Categories are mutually exclusive by classification order. They are approximate diagnostic groupings inferred from final recorded generation failure notes in `all_file_results.csv`, not manually verified semantic root causes.

A.14 Generation Failure-Mode Counts

Table A.14 reproduces the diagnostic generation failure-mode counts used in Chapter 5. The table is included here as an audit version of the main failure-mode table and should be read together with the classification rules in Table A.13.

Table A.14: Diagnostic generation failure modes for the main matched routes

Route	Pipeline	Failed n	Import	Interface	Assertion	Invalid	Runtime	Timeout	Other
Qwen 27B	Repository-aware	56	15	8	16	0	7	2	8
Qwen 27B	Generic baseline	95	0	11	37	0	21	2	24
Qwen 9B	Repository-aware	261	38	30	97	7	41	6	42
Qwen 9B	Generic baseline	330	8	35	114	2	60	10	101
Qwen 9B AWQ	Repository-aware	276	27	30	106	17	50	2	44
Qwen 9B AWQ	Generic baseline	383	11	37	145	5	61	12	112

Notes: Categories are mutually exclusive by classification order. They are diagnostic groupings inferred from the final recorded generation failure notes and should not be interpreted as manually verified semantic root causes. Values are derived from `all_file_results.csv` using the grouping rules in Table A.13; aggregate failure counts are consistent with `generation_failure.summary.csv`.

A.15 Chapter 5 Figure Source Data

Table A.15 summarises the source data used for the Chapter 5 figures. Exact numeric values are either reproduced in the corresponding appendix tables or summarised directly in this table.

Table A.15: Chapter 5 figure source data

Figure	Source data	Plotted values / audit location
Figure 5.1	main_pipeline_comparison.csv; condition_summary.csv	Plots final pass rate, target-normalised combined coverage and viable mutation score for the six matched generic-baseline versus repository-aware LazyTest routes. Exact values are reproduced in Table A.4.
Figure 5.3	secondary_backend_summary.csv; condition_summary.csv	Plots Qwen 27B prompt variants: base, strict mutation-aware and softened mutation-aware prompts, with and without prompt repetition. Exact values are reproduced as the Qwen 27B rows in Table A.8.
Figure 5.2	secondary_backend_summary.csv; condition_summary.csv	Plots selected secondary routes: Pynguin 60.58/71.52/40.65, Gemini 90.65/81.11/72.22, Qwen 27B base repeated 96.68/80.93/70.60, Qwen 27B softened mutation-aware repeated 93.27/81.96/74.23, Devstral AWQ base repeated 78.41/80.75/71.35, and Qwen3-Coder-Next AWQ base repeated 89.86/79.76/70.09, where each triplet is final pass / evaluated combined coverage / viable mutation score in percent.
Figure 5.4	condition_summary.csv	Plots matched average final pass rate for full and AWQ Qwen route pairs: Qwen 27B full 92.10%, Qwen 27B AWQ 90.42%, Qwen 9B full 66.38%, and Qwen 9B AWQ 62.24%. Supporting coverage and mutation values are provided in Table A.9.

Notes: Values are rounded to two decimal places, matching the presentation convention used in Chapter 5. The figures are visual summaries; the tables should be treated as the source of exact reported values.

A.16 Aggregation Caveats

Table A.16 records caveats relevant to the Chapter 5 aggregation. These caveats do not invalidate the evaluation, but they define how the reported metrics should be interpreted.

Table A.16: Aggregation caveats relevant to Chapter 5

Caveat	Observed issue	Treatment in this dissertation
Initial pass-rate construction	The generation metrics do not contain a separate explicit first-attempt status column for all routes. The aggregate therefore infers initial pass rate from final ok rows with attempts ≤ 1 .	Initial pass rate is reported as an inferred first-attempt proxy. Final pass rate remains the main directly interpretable executability metric.
Different metric denominators	Pass rate, coverage and mutation score do not use the same denominator. Coverage and mutation require accepted generated tests to remain executable in later post-processing stages.	Chapter 5 and Tables A.2, A.6 and A.12 report denominators explicitly.
Coverage fragility	Some tests that passed during generation later failed or timed out during coverage recalculation.	Coverage failures are retained as denominator evidence rather than silently ignored. Target-normalised coverage treats missing or failed coverage as zero over the full target set.
Mutation timeout retry artefacts	Retry-mutation artefacts are missing for some routes, including Gemini and several generic-baseline routes. In addition, the aggregate retry columns are not used as the basis for substantive mutation-score claims.	Timeout-retry outputs are treated as diagnostic artefacts only. They are not merged into the primary viable mutation-score denominator. Exact retry-timeout claims should be supported by the retry script or command log if retained in the main text.
Manifest mismatch warning	One Qwen 9B AWQ repeated strict mutation-aware run produced a warning that <code>target_module.py</code> appeared in generation and coverage rows but not in the target manifest.	The fixed 1,144-file target set remains the authoritative denominator. Unmatched rows are treated as aggregation warnings and not as additional case-study targets.
Route comparability	The 37 recorded conditions do not form a single flat model leaderboard because they differ by prompt, prompt repetition, backend, model family, quantisation and diagnostic availability.	Chapter 5 organises results by analysis role: main matched pipeline comparison, secondary routes, prompt variants, AWQ route checks, runtime and failure modes.
Displayed route labels	Appendix tables use shortened display labels such as <code>qwen.27b/run_7</code> rather than full internal folder names.	The shortened labels improve readability only. Full internal condition identifiers are retained in the source CSV artefacts and mapped through Table A.3.

Appendix B

Implementation Artefacts

This appendix provides selected implementation excerpts for the LazyTest artefacts discussed in Chapters ?? and 4. The listings are deliberately abridged: they show the mechanisms needed to substantiate the implementation claims without reproducing the full codebase. The complete project source, commit history and generated artefacts are retained in the project repository.

B.1 Implementation Artefact Index

Table B.1 summarises the implementation listings included in this appendix.

Table B.1: Implementation artefact index

Listing	Source artefact	Purpose
Listing B.1	lazytest-generator.yml	Pull-request-facing GitHub Actions workflow: ChatOps trigger, command parsing, backend selection and PR reporting.
Listing B.2	generate_tests_qwen_3.5_fixed_mut.py	Local target discovery and AST-based extraction of bounded target-symbol context.
Listing B.3	generate_tests_qwen_3.5_fixed_mut.py	Deterministic import handling, import stripping and controlled target_module loading.
Listing B.4	generate_tests_qwen_3.5_fixed_mut.py	Bounded Generate–Validate–Repair loop using pytest feedback.
Listing B.5	generate_baseline.py	Generic target-file baseline used for the matched comparison against repository-aware LazyTest.
Listing B.6	recalc_cov.py	Separate statement/branch coverage recalculation with strict failure handling.
Listing B.7	mutation_score.py	Primary mutation-evaluation procedure using isolated repository copies and Cosmic Ray.
Listing B.8	retry_mutation_timeouts.py	Diagnostic timeout-retry pass for primary mutation timeout cases.
Listing B.9	generate_tests_gemini.py	Gemini API backend as a substitution inside the same repository-aware LazyTest structure.
Listing B.10	generate_tests_qwen_3.5_fixed_mut.py; generate_tests_devstral.py	Route-level configuration evidence for model selection, prompt variants and backend-specific settings.

B.2 GitHub Actions ChatOps Workflow

Listing B.1 shows the pull-request-facing GitHub Actions layer. This workflow is the CI integration layer of LazyTest. It should be distinguished from the local empirical evaluation scripts used to generate the Chapter 5 results.

Listing B.1: Abridged GitHub Actions ChatOps workflow

```
name: LazyTest Generator

on:
```

```

issue_comment:
  types: [created]

permissions:
  contents: read

jobs:
  parse-command:
    if: >
      github.event.issue.pull_request &&
      contains(github.event.comment.body, '@lazytest-gen')

    runs-on: ubuntu-latest

    outputs:
      should_run: ${ steps.parse.outputs.should_run }
      generator: ${ steps.parse.outputs.generator }
      pynguin_args_json: ${ steps.parse.outputs.pynguin_args_json }
      commit_mode: ${ steps.parse.outputs.commit_mode }

    steps:
      - name: Parse LazyTest command
        id: parse
        shell: python
        env:
          COMMENT_BODY: ${ github.event.comment.body }
        run: |
          import json
          import os
          import re
          import shlex

          comment = os.environ.get("COMMENT_BODY", "")

          def write_output(name: str, value: str) -> None:
            value = str(value).replace("\n", " ").replace("\r", " ")
            with open(os.environ["GITHUB_OUTPUT"], "a", encoding="utf-8") as f:
              f.write(f"{name}={value}\n")

          match = re.search(
            r"(?im)^\s*@lazytest-gen(?:\s+(.))*?\s*$",
            comment,
          )

          if not match:
            write_output("should_run", "false")
            write_output("generator", "pynguin")
            write_output("pynguin_args_json", "[]")
            write_output("commit_mode", "passing")
            raise SystemExit(0)

          raw_args = (match.group(1) or "").strip()

          try:

```

```

        tokens = shlex.split(raw_args)
    except ValueError:
        tokens = raw_args.split()

    allowed_generators = {
        "pynguin",
        "chatgpt",
        "gemini",
        "pynguin-chatgpt",
        "pynguin-gemini",
        "local-llm",
        "pynguin-local-llm",
    }

    generator = "pynguin"
    commit_mode = "passing"
    pynguin_args = []
    ignored_args = []

    # Parse --generator, --commit-mode and a controlled subset of
    # Pynguin arguments from the pull-request comment.
    # Unknown generator values fall back to Pynguin.

generate-tests:
  needs: parse-command
  if: needs.parse-command.outputs.should_run == 'true'
  runs-on: ubuntu-latest

permissions:
  contents: write
  pull-requests: write
  issues: write

steps:
  # The workflow checks PR metadata, checks out the PR head,
  # detects changed Python files, dispatches the selected generator,
  # validates generated tests with pytest, uploads artefacts and
  # reports the result back to the pull request.

```

B.3 Target Discovery and AST-Based Context Extraction

Listing B.2 shows the local target discovery and bounded AST context extraction used by the repository-aware generation scripts. This supports the dissertation’s claim that LazyTest does not simply pass a target file to the model, but extracts a compact set of public symbols and signatures for the prompt.

Listing B.2: Target discovery and AST-based symbol extraction

```
def find_py_files(repo_path):
    py_files = []
    for root, dirs, files in os.walk(repo_path):
        skip = {'.git', '__pycache__', 'venv', '.venv',
                'node_modules', 'build', 'dist', 'docs', 'setup.py'}
        dirs[:] = [d for d in dirs if d not in skip]

        for f in files:
            if f == "__init__.py" or not f.endswith(".py"):
                continue

            full = os.path.join(root, f)
            norm = full.replace("\\", os.path.sep)

            if os.path.sep + "tests" + os.path.sep in norm:
                continue

            py_files.append(full)

    return sorted(py_files)

def get_symbols_with_signatures(tree):
    sigs = []

    for node in tree.body:
        if isinstance(node, ast.FunctionDef) and not node.name.startswith('_') and node.name != 'main':
            args = []

            for arg in node.args.args:
                if arg.annotation:
                    try:
                        args.append(f"{arg.arg}: {ast.unparse(arg.annotation)}")
                    except Exception:
                        args.append(arg.arg)
                else:
                    args.append(arg.arg)

            ret = ""

            if node.returns:
                try:
                    ret = f" -> {ast.unparse(node.returns)}"
                except Exception:
                    pass

            sigs.append(f"{node.name} ({', '.join(args)}){ret}")

    elif isinstance(node, ast.ClassDef) and not node.name.startswith('_'):
        sigs.append(f"class {node.name}")
```

```

    return sigs[:50]

# Inside per-file generation:
tree = ast.parse(source_code)
sigs = get_symbols_with_signatures(tree)
symbols_str = "\n".join(sigs) if sigs else "No top-level functions or classes found."

base_prompt = f"""
TESTING RULES:
- Test ONLY these explicitly defined symbols and respect their signatures:
{symbols_str}
...
"""

```

B.4 Deterministic Import Handling and Output Sanitisation

Listing B.3 shows the import-control mechanism used by the repository-aware pipeline. Model-generated import lines are stripped, and a deterministic preamble loads the target module by path, registers it under the expected module name and exposes it as `target_module`.

Listing B.3: Import stripping and deterministic target-module preamble

```

def strip_llm_imports(code: str) -> str:
    lines = code.splitlines()
    cleaned = []
    skip_next_blank = False

    for line in lines:
        stripped = line.strip()

        if stripped.startswith("import ") or stripped.startswith("from "):
            skip_next_blank = True
            continue

        if skip_next_blank and stripped == "":
            skip_next_blank = False
            continue

        skip_next_blank = False
        cleaned.append(line)

    return "\n".join(cleaned)

```

```

def build_preamble(repo_path: str, src_path: str, module_name: str) -> str:
    parts = module_name.split(".")
    package_name = ".".join(parts[:-1])

    lines = [
        "import sys, os, types, importlib.util",
        "import pytest",
        "import math, base64, string, re, collections, itertools",
        "import io",
        "from io import StringIO",
        "from contextlib import redirect_stdout, redirect_stderr",
        "from unittest.mock import MagicMock, patch, Mock",
        "",
        f"_REPO = r'{repo_path}'",
        "if _REPO not in sys.path:",
        "    sys.path.insert(0, _REPO)",
        "",
    ]

    for i in range(1, len(parts)):
        pkg = ".".join(parts[:i])
        pkg_path = os.path.join(repo_path, *parts[:i]).replace("\\", "/")

        lines += [
            f"if '{pkg}' in sys.modules:",
            f"    _file = getattr(sys.modules['{pkg}'], '__file__', '')",
            f"    if _file and 'generated_tests' in _file:",
            f"        del sys.modules['{pkg}']",
            f"if '{pkg}' not in sys.modules:",
            f"    _pkg = types.ModuleType('{pkg}']",
            f"    _pkg.__path__ = [r'{pkg_path}']",
            f"    _pkg.__package__ = '{pkg}'",
            f"    sys.modules['{pkg}'] = _pkg",
        ]

    lines += [
        "",
        f"for _key in ['{module_name}', 'target_module', '{parts[-1]}']:",
        f"    sys.modules.pop(_key, None)",
        "",
        f"_spec = importlib.util.spec_from_file_location('{module_name}', r'{src_path}']",
        f"_mod = importlib.util.module_from_spec(_spec)",
        f"_mod.__package__ = '{package_name}'",
        f"_mod.__spec__ = _spec",
        f"sys.modules['{module_name}'] = _mod",
        f"sys.modules['target_module'] = _mod",
        "",
        "try:",
        f"    _spec.loader.exec_module(_mod)",
        "except Exception as _import_err:",
        f"    raise RuntimeError(f'Failed to import {module_name}:'",
        f"    {type(_import_err).__name__}: {_import_err}]",
        "",
        f"target_module = _mod",
    ]

```

```

        "globals().update({k: v for k, v in vars(_mod).items() if not k.startswith('_')})",
    ]

    return "\n".join(lines)

# Inside per-file generation:
clean_code = strip_llm_imports(test_code)
preamble = build_preamble(repo_path, src, module_name)
final_test_code = preamble + "\n\n" + clean_code

```

B.5 Bounded Generate–Validate–Repair Loop

Listing B.4 shows the bounded execution-aware repair loop. The generated test is immediately run with `pytest`. If it fails, the traceback is fed back into the next prompt. Failed final test files are logged and removed from the output directory.

Listing B.4: Bounded pytest validation and repair loop

```

MAX_RETRIES = 5

while attempts < MAX_RETRIES:
    attempts += 1

    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {"role": "system",
             "content": "You are a Python QA engineer. Output only code. No reasoning, no
explanation."},
            {"role": "user", "content": current_prompt},
        ],
        temperature=0.8 + max(0, attempts - 1) * 0.01,
        top_p=0.95,
        max_tokens=16384,
        timeout=1200,
        extra_body={
            "top_k": 20,
            "min_p": 0.0,
            "repetition_penalty": 1.0,
            "chat_template_kwargs": {"enable_thinking": False},
        },
    )

    llm_output = response.choices[0].message.content
    test_code = extract_python_code(llm_output)

    if not test_code.strip() or "def test_" not in test_code:
        notes = "No valid tests generated"

```

```

        current_prompt = (
            base_prompt
            + "\n\nYOUR PREVIOUS RESPONSE WAS INVALID. "
            + "You must generate valid python code containing at least one "
            + "'def test_...' function."
        )
        continue

clean_code = strip_llm_imports(test_code)
preamble = build_preamble(repo_path, src, module_name)
final_test_code = preamble + "\n\n" + clean_code

with open(out_file, "w", encoding="utf-8") as wf:
    wf.write(final_test_code)

env = os.environ.copy()
paths_to_add = [src_dir, repo_path] if os.path.isdir(src_dir) else [repo_path]
env["PYTHONPATH"] = os.pathsep.join(paths_to_add) + os.pathsep + env.get("PYTHONPATH", "")

cmd = [
    sys.executable, "-m", "pytest", out_file,
    "--timeout=20",
    "-vv",
    "--tb=short",
    "-p", "no:cacheprovider",
    "--disable-warnings",
]

try:
    run_res = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        env=env,
        cwd=repo_path,
        timeout=30,
    )

    if run_res.returncode == 0:
        status = "ok"
        notes = f"Passed on attempt {attempts}"
        break

    error_output = (run_res.stdout + run_res.stderr).strip()[-15000:]

except subprocess.TimeoutExpired:
    error_output = "Subprocess Timeout: The generated test caused an infinite loop or deadlock."
    "

    notes = "TimeoutExpired"
    run_res = None

    current_prompt = f"""
[FIX ATTEMPT {attempts}/{MAX_RETRIES}] Your previous pytest file failed.
Analyze the traceback and output a fully fixed version.

```



```

Target source code:
{source_code}

Your test code that failed:
{test_code}

Pytest traceback:
{error_output}
"""

if status != "ok":
    with open(os.path.join(out_root, "failed_tests_log.txt"), "a", encoding="utf-8") as log_f:
        log_f.write(f"FAIL: {src}\n")
        log_f.write(f"Notes: {notes}\n")
        log_f.write(f"Full Pytest Output:\n{error_output}\n")

    if os.path.exists(out_file):
        os.remove(out_file)

return (repo_name, src, status, elapsed, len(test_code), test_code.count("def test_"), attempts,
        notes)

```

B.6 Generic Target-File Baseline

Listing B.5 shows the generic baseline used in the matched comparison. It deliberately omits LazyTest’s repository-aware AST-symbol prompt, deterministic import preamble and import stripping, while retaining bounded pytest validation and repair accounting.

Listing B.5: Generic target-file baseline structure

```

# Generic baseline LLM test generation script.
#
# Purpose:
# - Simple target-file baseline for comparison against repository-aware LazyTest.
# - No AST symbol extraction.
# - No deterministic import preamble.
# - No import stripping.
# - No mutation-aware prompt.
# - No repository-specific prompt engineering.
# - No prompt repetition.
#
# Behaviour:
# - For each Python source file, ask the LLM to generate pytest tests.
# - Run the generated tests with pytest.
# - If pytest passes, save the test file.
# - If pytest fails, send the previous test file and traceback back to the model.
# - Maximum 5 attempts per source file.

```

```

# - If all attempts fail, no final test file is kept.

MAX_RETRIES = 5

def build_base_prompt(module_name, source_path, source_code, uses_src_layout):
    src_note = (
        "- If the repository has a src/ directory, that src/ directory will also be on PYTHONPATH."
        if uses_src_layout
        else "- No src/ layout is assumed for this file."
    )

    return f"""
You are generating pytest unit tests for a Python source file.

Output rules:
- Output raw Python code only.
- Do not use markdown fences.
- Do not explain your answer.
- Do not reimplement source functions or classes.
- Prefer deterministic tests.
- Use pytest.

Execution context:
- The tests will be run from the repository root.
- The repository root will be on PYTHONPATH.
{src_note}
- The source file path relative to the repository root is: {source_path}
- The module under test should be imported as: {module_name}

Use this import pattern:

import importlib
target_module = importlib.import_module("{module_name}")

Then call functions and classes as target_module.<name>.

Source code:
{source_code}
"""

def run_pytest(test_file: str, repo_path: str) -> tuple[bool, str]:
    env = os.environ.copy()
    env["PYTHONPATH"] = pythonpath_for_repo(repo_path) + os.pathsep + env.get("PYTHONPATH", "")

    cmd = [
        sys.executable,
        "-m",
        "pytest",
        test_file,
        "--timeout=20",
        "--tb=short",
        "-q",
        "-p",
        "no:cacheprovider",
    ]

```

```

        "--disable-warnings",
    ]

    result = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        cwd=repo_path,
        env=env,
        timeout=PYTEST_TIMEOUT_SECONDS,
    )

    output = (result.stdout + "\n" + result.stderr).strip()
    return result.returncode == 0, output

# Failed temporary attempts are deleted. Passing attempts are moved to
# the final output path. If all attempts fail, the final test file is not kept.

```

B.7 Structural Coverage Recalculation

Listing B.6 shows the separate coverage post-processing step. It reruns accepted generated tests, enables branch coverage, extracts statement, branch and combined coverage fields from Coverage.py JSON, and does not credit coverage when the rerun fails under the strict policy.

Listing B.6: Separate statement and branch coverage recalculation

```

PYTEST_TIMEOUT_SECONDS = 10
SUBPROCESS_TIMEOUT_SECONDS = 45

# If True:
# - if pytest fails, coverage is not counted
# - file_cov_pct is set to 0.0
ONLY_COUNT_COVERAGE_IF_PYTEST_PASSES = True

EXTRA_COLUMNS = [
    "branch_cov_pct",
    "combined_cov_pct",
    "covered_branches",
    "num_branches",
    "missing_branches",
    "partial_branches",
    "pytest_exit_code",
    "coverage_eval_status",
]

def extract_target_metrics(summary):
    """
    With --cov-branch enabled:

```

```

- percent_statements_covered = statement/file coverage
- percent_branches_covered = branch coverage
- percent_covered = combined statement + branch coverage
"""
statement_cov = summary.get(
    "percent_statements_covered",
    summary.get("percent_covered", 0.0),
)

branch_cov = summary.get("percent_branches_covered", "")
combined_cov = summary.get("percent_covered", "")

return {
    "statement_cov": round_or_blank(statement_cov),
    "branch_cov": round_or_blank(branch_cov),
    "combined_cov": round_or_blank(combined_cov),
    "covered_branches": int_or_blank(summary.get("covered_branches", "")),
    "num_branches": int_or_blank(summary.get("num_branches", "")),
    "missing_branches": int_or_blank(summary.get("missing_branches", "")),
    "partial_branches": int_or_blank(summary.get("num_partial_branches", "")),
}

cmd = [
    "pytest",
    test_file,
    "-o", "addopts=",
    f"--cov={REPO_ROOT}",
    "--cov-branch",
    f"--cov-report=json:{cov_json}",
    "--import-mode=importlib",
    "-q",
    f"--timeout={PYTEST_TIMEOUT_SECONDS}",
    "--disable-warnings",
    "-p", "no:cacheprovider",
]

result = subprocess.run(
    cmd,
    env=env,
    capture_output=True,
    text=True,
    timeout=SUBPROCESS_TIMEOUT_SECONDS,
    cwd=REPO_ROOT,
)

if result.returncode != 0:
    row = append_note(row, f"pytest failed during coverage eval, exit={result.returncode}")

if ONLY_COUNT_COVERAGE_IF_PYTEST_PASSES:
    row[col["file_cov_pct"]] = 0.0
    return row + blank_extra(
        pytest_exit_code=result.returncode,
        eval_status="pytest_failed_coverage_not_counted",
    )

```

```

if not os.path.exists(cov_json):
    row[col["file_cov_pct"]] = 0.0
    row = append_note(row, "Coverage JSON not created")
    return row + blank_extra(eval_status="coverage_json_missing")

with open(cov_json, "r", encoding="utf-8") as f:
    data = json.load(f)

target_fdata = find_target_file_coverage(data, rel_path)

if target_fdata is None:
    row[col["file_cov_pct"]] = 0.0
    row = append_note(row, f"Target file not found in coverage JSON: {rel_path}")
    return row + blank_extra(eval_status="target_file_missing_from_coverage_json")

metrics = extract_target_metrics(target_fdata.get("summary", {}))
row[col["file_cov_pct"]] = metrics["statement_cov"]

```

B.8 Mutation Evaluation

Listing B.7 shows the primary mutation-evaluation procedure. The script copies the repository into a temporary directory, validates the generated test first, writes a Cosmic Ray configuration, executes mutation testing, parses the resulting SQLite database and reports viable mutation score as killed over killed plus survived mutants.

Listing B.7: Primary Cosmic Ray mutation evaluation

```

DEFAULT_BASELINE_TIMEOUT = 60
DEFAULT_COSMIC_INIT_TIMEOUT = 60
DEFAULT_COSMIC_EXEC_TIMEOUT = 300
DEFAULT_PER_TEST_TIMEOUT = 20

def calculate_scores(*, killed: int, survived: int, incompetent: int,
                    timeout: int, other: int) -> dict[str, Any]:
    viable = killed + survived
    total = killed + survived + incompetent + timeout + other

    if viable > 0:
        score_viable = round((killed / viable) * 100.0, 2)
    else:
        score_viable = ""

    if total > 0:
        score_raw = round((killed / total) * 100.0, 2)
    else:
        score_raw = ""

    return {

```

```

        "viable_mutants": viable,
        "total_mutants": total,
        "mutation_score_viable": score_viable,
        "mutation_score_raw": score_raw,
    }

def evaluate_task(task, *, repo_root, cosmic_ray_bin, baseline_timeout,
                 cosmic_init_timeout, cosmic_exec_timeout, per_test_timeout,
                 keep_temp_on_failure, extra_rewrite_paths):
    tmp_context = tempfile.TemporaryDirectory(prefix="lazytest_mutation_")
    tmpdir = Path(tmp_context.name)
    repo_copy_dir = tmpdir / REPO_NAME

    shutil.copytree(
        repo_root,
        repo_copy_dir,
        ignore=shutil.ignore_patterns(
            ".git", ".pytest_cache", "__pycache__", ".mypy_cache",
            ".ruff_cache", ".venv", "venv",
        ),
    )

    isolated_source_file = repo_copy_dir / task["source_file"]
    isolated_test_file = tmpdir / Path(task["test_file"]).name

    test_content = test_abs.read_text(encoding="utf-8", errors="replace")
    test_content = test_content.replace(str(repo_root), str(repo_copy_dir))
    isolated_test_file.write_text(test_content, encoding="utf-8")

    env = os.environ.copy()
    env["PYTHONPATH"] = f"{repo_copy_dir}{os.pathsep}{env.get('PYTHONPATH', '')}"

    baseline_cmd = [
        sys.executable,
        "-m",
        "pytest",
        str(isolated_test_file),
        "-q",
        "--disable-warnings",
        "-p", "no:cacheprovider",
        "-p", "no:doctest",
        "-c", "/dev/null",
        f"--timeout={per_test_timeout}",
        "--tb=short",
    ]

    rc, stdout, stderr, elapsed = run_subprocess(
        baseline_cmd,
        env=env,
        cwd=repo_copy_dir,
        timeout=baseline_timeout,
        label=f"baseline {task['source_file']}",
    )

```

```

if rc != 0:
    return finish_result(
        row,
        status="baseline_failed",
        notes=f"Baseline test failed. stdout={stdout[-300:]} stderr={stderr[-300:]}",
        started_at=started_at,
    )

db_path = repo_copy_dir / "session.sqlite"
toml_path = repo_copy_dir / "cr.toml"

toml_content = f"""
[cosmic-ray]
module-path = {json.dumps(str(isolated_source_file))}
timeout = {float(per_test_timeout)}
test-command = {json.dumps(test_cmd)}
excluded-modules = []

[cosmic-ray.distributor]
name = "local"
"""

toml_path.write_text(toml_content, encoding="utf-8")

rc_init, stdout_init, stderr_init, elapsed_init = run_subprocess(
    [cosmic_ray_bin, "init", str(toml_path), str(db_path)],
    env=env,
    cwd=repo_copy_dir,
    timeout=cosmic_init_timeout,
    label=f"cosmic-ray init {task['source_file']}",
)

rc_exec, stdout_exec, stderr_exec, elapsed_exec = run_subprocess(
    [cosmic_ray_bin, "exec", str(toml_path), str(db_path)],
    env=env,
    cwd=repo_copy_dir,
    timeout=cosmic_exec_timeout,
    label=f"cosmic-ray exec {task['source_file']}",
)

parsed = parse_cosmic_ray_db(db_path)

row["mutants_killed"] = parsed.get("killed", 0)
row["mutants_survived"] = parsed.get("survived", 0)
row["mutants_incompetent"] = parsed.get("incompetent", 0)
row["mutants_timeout"] = parsed.get("timeout", 0)
row["mutants_other"] = parsed.get("other", 0)

scores = calculate_scores(
    killed=int(row["mutants_killed"] or 0),
    survived=int(row["mutants_survived"] or 0),
    incompetent=int(row["mutants_incompetent"] or 0),
    timeout=int(row["mutants_timeout"] or 0),
    other=int(row["mutants_other"] or 0),

```

```

)
row.update(scores)

return finish_result(
    row,
    status="mutation_processed",
    notes=f"Cosmic Ray processed. score_viable={row['mutation_score_viable']}",
    started_at=started_at,
)

```

B.9 Mutation Timeout Retry

Listing B.8 shows the diagnostic timeout-retry script. This retry pass is not merged into the primary viable mutation-score denominator; it is retained as a diagnostic artefact for timeout analysis.

Listing B.8: Diagnostic mutation-timeout retry pass

```

DEFAULT_SOURCE_CSV_NAME = "mutation_cosmic_ray.csv"
DEFAULT_RETRY_CSV_NAME = "mutation_cosmic_ray_timeout_retry.csv"
DEFAULT_TARGETS_NAME = "timeout_targets_retry.txt"

def collect_timeout_targets(*, run_dir: Path,
                           source_csv_name: str,
                           targets_name: str) -> tuple[Path, list[str]]:
    source_csv = run_dir / source_csv_name
    targets_path = run_dir / targets_name

    rows = read_csv(source_csv)

    targets = sorted(
        {
            row["source_file"]
            for row in rows
            if row.get("mutation_status") == "cosmic_exec_timeout"
            and row.get("source_file")
        },
        key=natural_key,
    )

    if targets:
        targets_path.write_text("\n".join(targets) + "\n", encoding="utf-8")
    else:
        if targets_path.exists():
            targets_path.unlink()

    return targets_path, targets

def run_retry_for_run(*, args, model: str,

```



```

        run_dir: Path,
        targets_path: Path) -> int:
    throwaway_combined_name = f"_mutation_retry_latest_{model}_{run_dir.name}.csv"

    cmd = [
        sys.executable,
        str(args.mutation_script),
        "--generated-root", str(args.generated_root),
        "--repo-root", str(args.repo_root),
        "--models", model,
        "--run-pattern", run_dir.name,
        "--targets-file", str(targets_path),
        "--max-workers", str(args.max_workers),
        "--cosmic-exec-timeout", str(args.cosmic_exec_timeout),
        "--per-test-timeout", str(args.per_test_timeout),
        "--save-every", str(args.save_every),
        "--progress-every", str(args.progress_every),
        "--out-csv-name", args.retry_csv_name,
        "--combined-csv-name", throwaway_combined_name,
    ]

    completed = subprocess.run(cmd, env=env)
    return completed.returncode

parser.add_argument(
    "--cosmic-exec-timeout",
    type=int,
    default=900,
    help="Cosmic Ray exec timeout in seconds for retry pass. Default: 900",
)

summary_payload = {
    "total_original_timeout_targets": total_targets,
    "failed_retry_invocations": failures,
    "retry_exec_timeout_s": args.cosmic_exec_timeout,
    "retry_workers": args.max_workers,
    "per_test_timeout_s": args.per_test_timeout,
    "runs": summary,
}

summary_json.write_text(json.dumps(summary_payload, indent=2), encoding="utf-8")

```

B.10 Gemini Backend

Listing B.9 shows the Gemini API route as a backend substitution inside the same repository-aware pipeline. The API key line is redacted and should not be reproduced in the dissertation.

Listing B.9: Gemini backend inside the repository-aware LazyTest pipeline

```

# LazyTest repository-aware test generation using Gemini.
#
# Pipeline:
# - Generate pytest tests for Python files in TheAlgorithms using Gemini API.
# - Preserve the repository-aware LazyTest methodology:
#   AST symbol extraction, deterministic import preamble, import stripping,
#   target_module.<name>(...) access, immediate pytest validation, and bounded
#   repair.
#
# It is not an autonomous coding-agent workflow.

MAX_WORKERS = 8
MAX_RETRIES = 5

BACKEND_NAME = "gemini-3.1-flash-lite-preview"
GEMINI_API_KEY = "<redacted>"
GEMINI_MODEL = "gemini-3.1-flash-lite-preview"
GEMINI_MAX_OUTPUT_TOKENS = 16384

SYSTEM_INSTRUCTION = (
    "You are a Python QA engineer. Output only code. "
    "No reasoning, no explanation."
)

def call_gemini(prompt: str, attempt: int):
    client = get_gemini_client()

    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=prompt,
        config=genai_types.GenerateContentConfig(
            system_instruction=SYSTEM_INSTRUCTION,
            max_output_tokens=GEMINI_MAX_OUTPUT_TOKENS,
        ),
    )

    return response

# The Gemini script reuses the same repository-aware mechanisms:
# - find_py_files(...)
# - get_symbols_with_signatures(...)
# - strip_llm_imports(...)
# - build_preamble(...)
# - immediate pytest validation
# - MAX_RETRIES bounded repair

response = call_gemini(current_prompt, attempts)
llm_output = get_gemini_text(response)
test_code = extract_python_code(llm_output)

clean_code = strip_llm_imports(test_code)
preamble = build_preamble(repo_path, src, module_name)
final_test_code = preamble + "\n\n" + clean_code

```

```

cmd = [
    sys.executable,
    "-m",
    "pytest",
    out_file,
    "--timeout=20",
    "-vv",
    "--tb=short",
    "-p",
    "no:cacheprovider",
    "--disable-warnings",
]

```

B.11 Route and Prompt Configuration Variants

Listing B.10 shows representative route-level configuration differences. Qwen-family routes used the same repository-aware pipeline with model, output-folder and prompt-variant changes. The Devstral route used the same local generation structure with a different OpenAI-compatible endpoint and a model-specific prompt configuration.

Listing B.10: Representative model and prompt route configuration

```

# Qwen route configuration.
MAX_WORKERS = 60

SCRIPT_DIR = Path(__file__).parent.resolve()
PROJECT_ROOT = SCRIPT_DIR.parent.parent
REPOS = [str(PROJECT_ROOT / "testing" / "repos_for_testing" / "TheAlgorithms")]

OUT_DIR = str(
    PROJECT_ROOT
    / "testing"
    / "generated_tests"
    / "qwen_3.5"
    / "run_9_no_repeat_mut_fixed_long_trace"
)

API_BASE_URL = "http://localhost:18000/v1"
MODEL_NAME = "Qwen/Qwen3.5-27B"

client = OpenAI(base_url=API_BASE_URL, api_key="not-needed")

# Prompt-variant evidence from the softened mutation-aware prompt.
base_prompt = f"""
...
- METRIC TARGETS: Maximize branch and line coverage, and prioritize a high Mutation Score.
- BEYOND COVERAGE: Write "strong" tests that detect logic changes (mutants).
...

```

```

"""

# Devstral route configuration.
MAX_WORKERS = 8

REPOS = [
    os.path.expanduser("<redacted>")
]

OUT_DIR = os.path.expanduser(
    "<redacted>"
)

API_BASE_URL = "http://localhost:8000/v1"
MODEL_NAME = "cyankiwi/Devstral-Small-2-24B-Instruct-2512-AWQ-4bit"

client = OpenAI(base_url=API_BASE_URL, api_key="not-needed")

# Devstral used the same overall repository-aware structure:
# target discovery, output sanitisation, import preamble, pytest validation
# and bounded repair, but with route-specific decoding/prompt settings.
response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {
            "role": "user",
            "content": (
                "You are a Python QA engineer. Output only code. "
                "No reasoning, no explanation.\n\n"
                + current_prompt
            ),
        }
    ],
    frequency_penalty=0.05,
    presence_penalty=0.0,
    temperature=0.15 + max(0, attempts - 1) * 0.025,
    max_tokens=16384,
    timeout=1200,
)

```

B.12 Implementation Evidence Summary

The listings in this appendix support three implementation boundaries used throughout the dissertation. First, the GitHub Actions workflow provides the pull-request-facing ChatOps integration, but it is not the source of the Chapter 5 empirical results. Secondly, the local repository-aware generation scripts implement bounded context extraction, deterministic import handling and pytest-based repair for the controlled TheAlgorithms/Python case study. Thirdly, the post-processing scripts recompute coverage

and mutation metrics in separate stages, using narrower denominators than the initial generation target set. This distinction is important because the evaluation claims in Chapter 5 depend on execution, coverage and mutation artefacts rather than on generated code alone.